



ITCS223

Introduction to Web Development

Project Phase II

Submitted by

6688069 Worasuda Ngamyingsurat

6688072 Kemjira Nugboon

6688099 Nuttanan Reamprasert

6688129 Patsatraporn Thongdeesakul

6688210 Thanaporn Pinsakul

Faculty of Information and Communication Technology

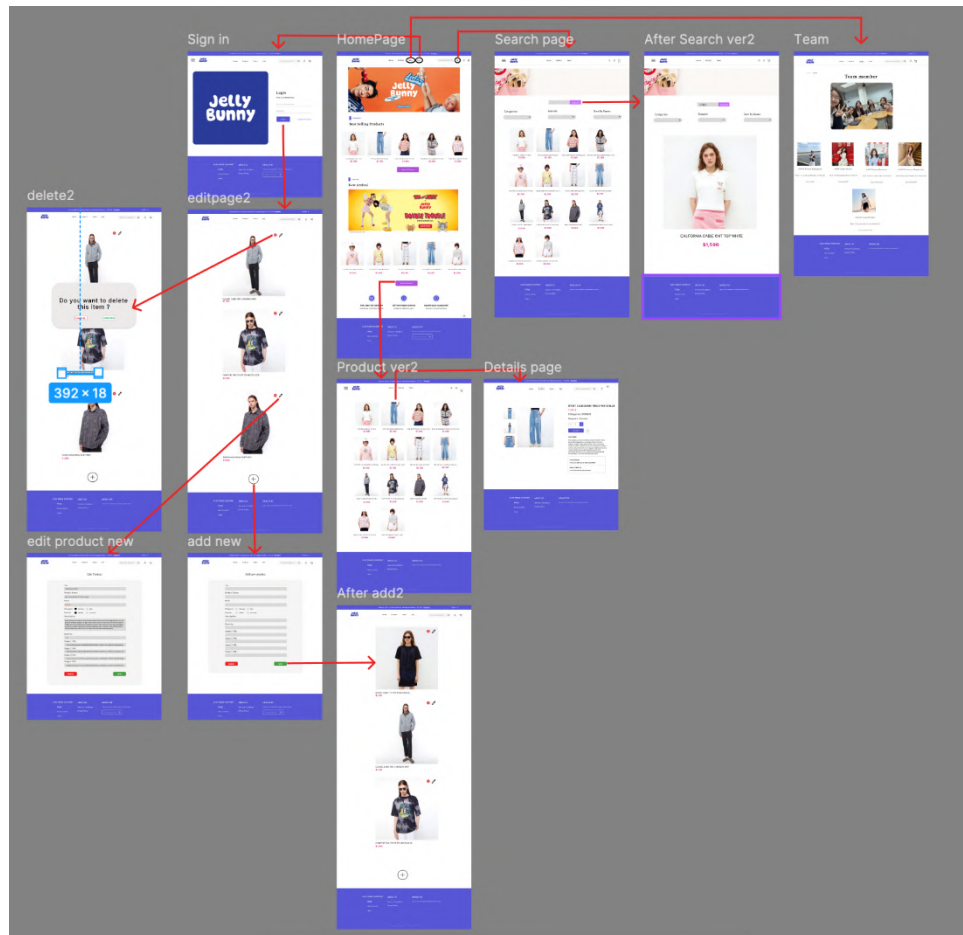
Mahidol University

Project Overview

In this group project, we are developing Jelly Bunny, an online shopping web application specializing in selling clothes, shoes, and bags. Building upon the design phase, the project focuses on implementing both the front-end and back-end components. The front-end is developed using HTML, CSS, and JavaScript, with all pages hosted on a dedicated front-end server. The design emphasizes user-friendly navigation, responsive layouts, and a visually appealing shopping experience.

On the back-end, we use Node.js to manage the database and provide web services, storing product information, user accounts, orders, and other relevant data. The web services handle user interactions such as browsing products, managing the shopping cart, and placing orders. The application includes several key pages, such as the Home Page with featured products, the Sign in Page for user login, the Product Page displaying available items, the Product Detail Page with detailed product information, and the Team Page introducing the development team. For administrative control, there are three pages: the Edit Product Page, Delete Product Page, and Add New Product Page, which allow administrators to manage the product catalog. We use Figma to create wireframes, prototypes, and flow diagrams, helping visualize the page structure, user journeys, and design consistency before development. This phase aims to integrate the front-end and back-end to deliver a seamless, interactive, and reliable online shopping experience.

Navigation Diagram of your web application



<https://www.figma.com/design/uKVDUsZ9rt39p223thySFq/Jelly-bunny?node-id=1-3&t=bTGAM0vncFnZo6Pb-1>

Details of your web application and code

frontend.js

```
const PORT = 3300;
const path = require("path");
const express = require("express");
const app = express();
const dotenv = require("dotenv");
dotenv.config();
const router = express.Router();
```

The frontend.js file will contain port 3300 which is the port for our front-end. However, our front-end will want to input the command which are

- npm install cors
- npm install express
- npm install nodemon
- npm install path
- npm install url

```
const cors = require("cors");

/* This is for specific origin and method */
let corsOptions = {
  origin: 'http://localhost:8000',
  methods: 'GET,POST,PUT,DELETE'
}
router.use(cors(corsOptions));
// router.use(cors());
```

“corsOption” will use for switch port between frontend and backend. Only GET, POST, PUT, and DELETE methods are permitted for cross-origin communication.

```
app.use("/", router);
app.use("/picture", express.static(path.join(__dirname, "picture")));
app.use("/css", express.static("css"));
app.use("/js", express.static(path.join(__dirname, "js")));

router.use(express.json());
router.use(express.urlencoded({ extended: true }));
app.use(express.static('public'));
```

In this section of the code, we set up routing for different pages of the web application and serve static files such as images, CSS, and JavaScript.

1. Serving Static Files
 - o The server serves images from the /picture folder.

- o CSS files are served from the /css folder.
- o JavaScript files are served from the /js folder.
- o Public files are served from the public folder.
- o This is done using `express.static()` to make these resources available to the frontend.

2. Middleware for Handling Requests

`router.use(express.json())` allows the server to accept JSON data in requests.

`router.use(express.urlencoded({ extended: true }))` allows the server to handle form submissions (URL-encoded data)

3. Setting Up Routes

`/homepage`: When a user visits `/`, `/homepage`, the server responds with the `homepage.html` file.

```
// Route for the Homepage
router.get("/", (req, res) => {
  console.log("--Connected to Homepage")
  res.sendFile(path.join(`${__dirname}/html/homepage.html`));
});

router.get("/homepage", (req, res) => {
  console.log("--Connected to Webpage")
  res.sendFile(path.join(`${__dirname}/html/homepage.html`));
});
```

`/sign-in`: When a user visits `/sign-in`, the server sends the `sign-in.html` file.

```
// Route for the Sign-in Page
router.get("/sign-in", (req, res) => {
  console.log("--Connected to Signin Page")
  res.sendFile(path.join(`${__dirname}/html/sign-in.html`));
});
```

`/team`: When a user visits `/team`, the server sends the `Team.html` file.

```
// Route for the Team Page
router.get("/team", (req, res) => {
  console.log("--Connected to Team Page")
  res.sendFile(path.join(`${__dirname}/html/Team.html`));
});
```

`/search`: When a user visits `/search`, the server sends the `search.html` file.

```
// Route for the Search Page
router.get('/search', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/search.html`))
  console.log("--Connected to Search Page")
});
```

/product: When a user visits /product, the server sends the Product.html file.

```
// Route for the Product Page
router.get('/product', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/Product.html`))
  console.log("--Connected to Product Page")
});
```

/add: When a user visits /add, the server sends the add.html file.

```
// Route for the Add Product Page
router.get('/add', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/add.html`))
  console.log("--Connected to Add Product Page")
});
```

/detail-page: When a user visits /detail-page, the server sends the detailpage.html file.

```
// Route for the Detail Page
router.get('/detail-page', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/detailpage.html`))
  console.log("--Connected to Detail Page")
});
```

/edit: When a user visits /edit, the server sends the edit-page.html file.

```
// Route for the Edit Page
router.get('/edit', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/edit-page.html`))
  console.log("--Connected to Edit Page")
});
```

/edit-product: When a user visits /edit-product, the server sends the edit.html file.

```
// Route for the Edit Product Page
router.get('/edit-product', (req, res) => {
  res.sendFile(path.join(`${__dirname}/html/edit.html`))
  console.log("--Connected to Edit Product Page") //
});
```

Each route logs a message in the console to confirm the connection.

```
app.listen(PORT, () => {
  console.log(`Server listening on port: ${PORT}`);
})
```

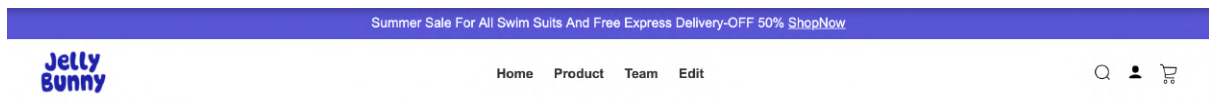
HTML PART

For every page, start with the **header part** containing this:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Jelly Bunny - Men's Fashion</title>
  <link rel="stylesheet" href="/css/add.css">
  <script src="/js/cookie.js"></script>
</head>
```

- `<meta charset="UTF-8">` – Sets the character encoding to UTF-8, supporting various languages.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` – Ensures the page is responsive and adjusts to device width.
- `<title>Jelly Bunny - Men's Fashion</title>` – Sets the title displayed on the browser tab.
- `<link rel="stylesheet" href="/css/add.css">` – Links an external CSS file for styling the page. The href attribute can point to any CSS file inside the /css folder.
- `<script src="/js/cookie.js"></script>` – Links an external JavaScript file for cookie handling or other functionality.

In the body part of every html will have this part as a **navigation bar**:

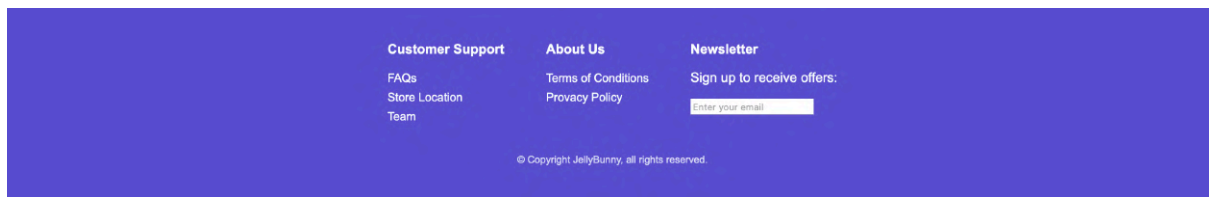


```
<body>
  <header>
    <div class="top-banner">
      Summer Sale For All Swim Suits And Free Express Delivery-OFF 50% <a href="#">ShopNow</a>
    </div>
    <nav class="navbar">
      <div class="logo">
        
      </div>
      <ul class="nav-links">
        <li><a href="/homepage">Home</a></li>
        <li><a href="/product">Product</a></li>
        <li><a href="/team">Team</a></li>
        <li><a href="/edit">Edit</a></li>
      </ul>
      <div class="nav-icons">
        <a href="/search"></a>
        <a href="/sign-in"></a>
        </a>
      </div>
    </nav>
  </header>
```

This HTML code creates the header section of a webpage, featuring a promotional top banner announcing a summer sale with a "Shop Now" link. Below the banner, a navigation bar is structured with a brand logo on the left, a set of navigation links (Home, Product, Team, Edit) in the center, and action icons (search, user profile, and cart) on the right. External images are used for the logo and icons, and all links are structured using anchor tags.

There is a minor mistake with an extra closing `</nav>` tag, but overall, the code builds a clean, functional header layout for easy navigation.

In the body part of every html will have this part as a **footer**:



```
<!--Footer-->
<footer>
  <div class="footer-container">
    <div class="footer-column">
      <h4>Customer Support</h4>
      <ul>
        <li><a href="#">FAQs</a></li>
        <li><a href="#">Store Location</a></li>
        <li><a href="#">Team</a></li>
      </ul>
    </div>
    <div class="footer-column">
      <h4>About Us</h4>
      <ul>
        <li><a href="#">Terms of Conditions</a></li>
        <li><a href="#">Provacy Policy</a></li>
      </ul>
    </div>
    <div class="footer-column news">
      <h4>Newsletter</h4>
      <p>Sign up to receive offers:</p>
      <input type="email" placeholder="Enter your email" />
    </div>
  </div>
  <p class="copyright">&copy; Copyright JellyBunny, all rights reserved.</p>
</footer>
```

This HTML code creates the footer section of a webpage, organized into three columns within a container. The first column, labeled "Customer Support," provides links to FAQs, store location, and the team page. The second column, titled "About Us," includes links to the terms and conditions and the privacy policy (noting a small typo in "Privacy Policy"). The third column offers a "Newsletter" signup area, inviting users to enter their email to receive special offers. Below the columns, a copyright notice states that all rights are reserved by JellyBunny. Overall, the footer provides important navigation links, policy information, and a way for users to stay connected with the brand.

- Team.html

Summer Sale For All Swim Suits And Free Express Delivery-OFF 50% ShopNow

JettyBunny

HomeProductTeamEdit

Q

Team Member





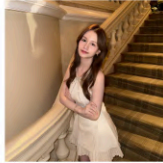
6688069 Worasuda Ngamyingsurat
Email: worasuda.nga@student.mahidol.edu
Line: [fai8181](https://line.me/tv/fai8181)



6688072 Kemjira Nugboon
Email: Kemjira.nug@student.mahidol.edu
Line: [yoke3174](https://line.me/tv/yoke3174)



6688099 Nuttanan Raamprasert
Email: Nuttanan.raa@student.mahidol.edu
Line: [pamnuttanan](https://line.me/tv/pamnuttanan)



6688129 Patsatrapom Thongdeesakul
Email: Patsatrapom.tho@student.mahidol.edu
Line: [0915462944](https://line.me/tv/0915462944)



6688210 Thanaporn Pinsakul
Email: thanaporn.pis@student.mahidol.edu
Line: [nongluktan123](https://line.me/tv/nongluktan123)

Customer Support

FAQs
Store Location
Team

About Us

Terms of Conditions
Privacy Policy

Newsletter

Sign up to receive offers:

© Copyright JettyBunny, all rights reserved.

```

<!-- Team Section -->
<div class="team-container">
  <div class="team-header">Team Member</div>

  <!-- Team Image -->
  

  <!-- Team Member Info -->
  <div class="team-info">
    <!-- Team Member 1 -->
    <div class="member">
      
      <div class="member-info">
        <div class="name">6688069 Worasuda Ngamyingsurat</div>
        <div class="email">Email: <a href="mailto:worasuda.nga@student.mahidol.edu">worasuda.nga@student.mahidol.edu</a></div>
        <div class="line">Line: fai8181</div>
      </div>
    </div>

    <!-- Team Member 2 -->
    <div class="member">
      
      <div class="member-info">
        <div class="name">6688072 Kemjira Nugboon</div>
        <div class="email">Email: <a href="mailto:Kemjira.nug@student.mahidol.edu">Kemjira.nug@student.mahidol.edu</a></div>
        <div class="line">Line: yoke3174</div>
      </div>
    </div>

    <!-- Team Member 3 -->
    <div class="member">
      
      <div class="member-info">
        <div class="name">6688099 Nuttanan Reamprasert</div>
        <div class="email">Email: <a href="mailto:Nuttanan.rea@student.mahidol.edu">Nuttanan.rea@student.mahidol.edu</a></div>
        <div class="line">Line: pamnuttanan</div>
      </div>
    </div>

    <!-- Team Member 4 -->
    <div class="member">
      
      <div class="member-info">
        <div class="name">6688129 Patsatraporn Thongdeesakul</div>
        <div class="email">Email: <a href="mailto:Patsatraporn.tho@student.mahidol.edu">Patsatraporn.tho@student.mahidol.edu</a></div>
        <div class="line">Line: 0915462944</div>
      </div>
    </div>

    <!-- Team Member 5 -->
    <div class="member">
      
      <div class="member-info">
        <div class="name">6688210 Thanaporn Pinsakul</div>
        <div class="email">Email: <a href="mailto:thanaporn.pis@student.mahidol.edu">thanaporn.pis@student.mahidol.edu</a></div>
        <div class="line">Line: nongluktan123</div>
      </div>
    </div>
  </div>
</div>

```

The provided HTML code structures the Team Section for a webpage, presenting detailed information about each team member. Below is a breakdown of the components:

1.Container (team-container):

The entire Team Section is encapsulated within a div element with the class team-container. This container holds all the elements of the team section together, ensuring they are properly aligned and grouped.

2.Header (team-header):

The team-header class defines the section's header, which contains the title "Team Member." This title appears at the top of the section, clearly introducing the team to the user.

3.Team Member Information (team-info):

The team-info section holds the individual cards for each team member. These cards display important details about each person on the team.

4. Profile Picture:

Each team member has a profile picture displayed through an img tag. The src attribute points to the image file for the member's photo, such as /picture/image 84.png for the first member. This image is shown alongside the member's other information.

5. Team Member Details:

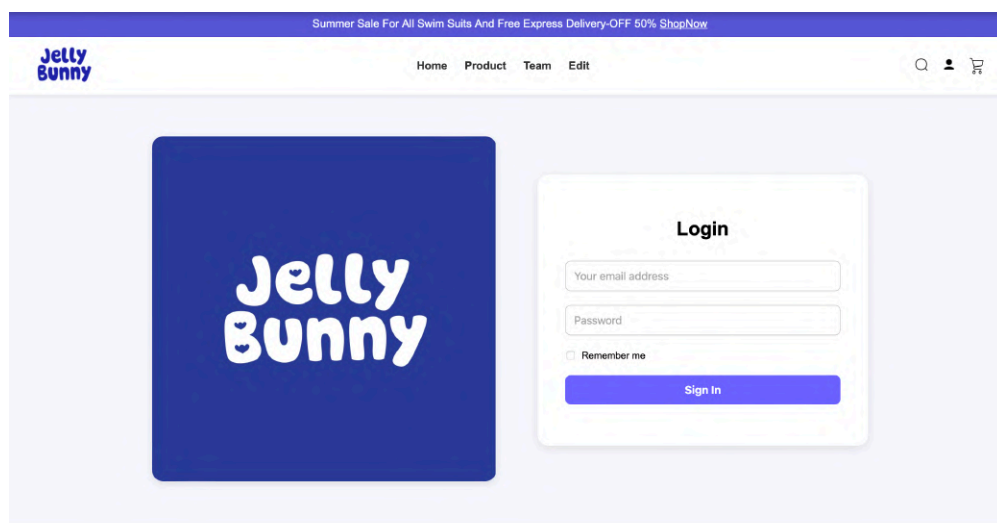
Name: The name of the team member is displayed inside a div with the class name.

Student ID: The student ID is shown with student-id

Email: A clickable email link (mailto:) is provided for each member, enabling users to contact them directly via email.

Line ID: The Line ID is shown for easy social media contact.

- Sign-in.html



```
<!-- Login Section -->
<div class="login-container">
  <!-- Logo (Big image) -->
  <div class="logo-container">
    
  </div>

  <div class="signin-signup">
    <div class="segment">
      <form id="signInForm" class="sign-in-form">
        <h2 class="title">Login</h2>
        <div class="form-row">
          <input type="text" class="form-control" name="username" id="username"
            placeholder="Your email address" required>
        </div>
        <div class="form-row">
          <input type="password" class="form-control" name="password" id="password" placeholder="Password"
            required>
        </div>
        <div class="checkbox">
          <input type="checkbox" id="remember-me">
          <label for="remember-me">Remember me</label>
        </div>
        <button class="sign-in button" type="button" onclick="signIn()">Sign In</button>
      </form>
      <!-- Display message -->
      <div id="message" class="message"></div>
    </div>
  </div>
</div>
```

This HTML code builds a Login Section for a webpage.

1. Main Container (login-container): Wraps all the login content together.

2. Logo Area (logo-container): Shows a brand logo using a file /picture/login.jpg, displayed at 500 pixels wide.
3. Sign-in Area (signin-signup): Contains the login form inside a segment box.
4. Login Form (signInForm):
 - Has a heading “Login” (h2 tag).
 - Contains two input fields:
 - Email address (type=”text”) where users enter their email.
 - Password (type=”password”) where users enter their password.
 - Includes a “Remember me” checkbox that users can tick if they want the site to remember their login details.
 - A Sign In button triggers a JavaScript function called signIn() when clicked.

Message Display (message): An empty div where messages (like login errors or success notifications) will appear after attempting to sign in.

```
<script>
function signIn() {
  const tx_username = document.getElementById("username").value;
  const tx_password = document.getElementById("password").value;

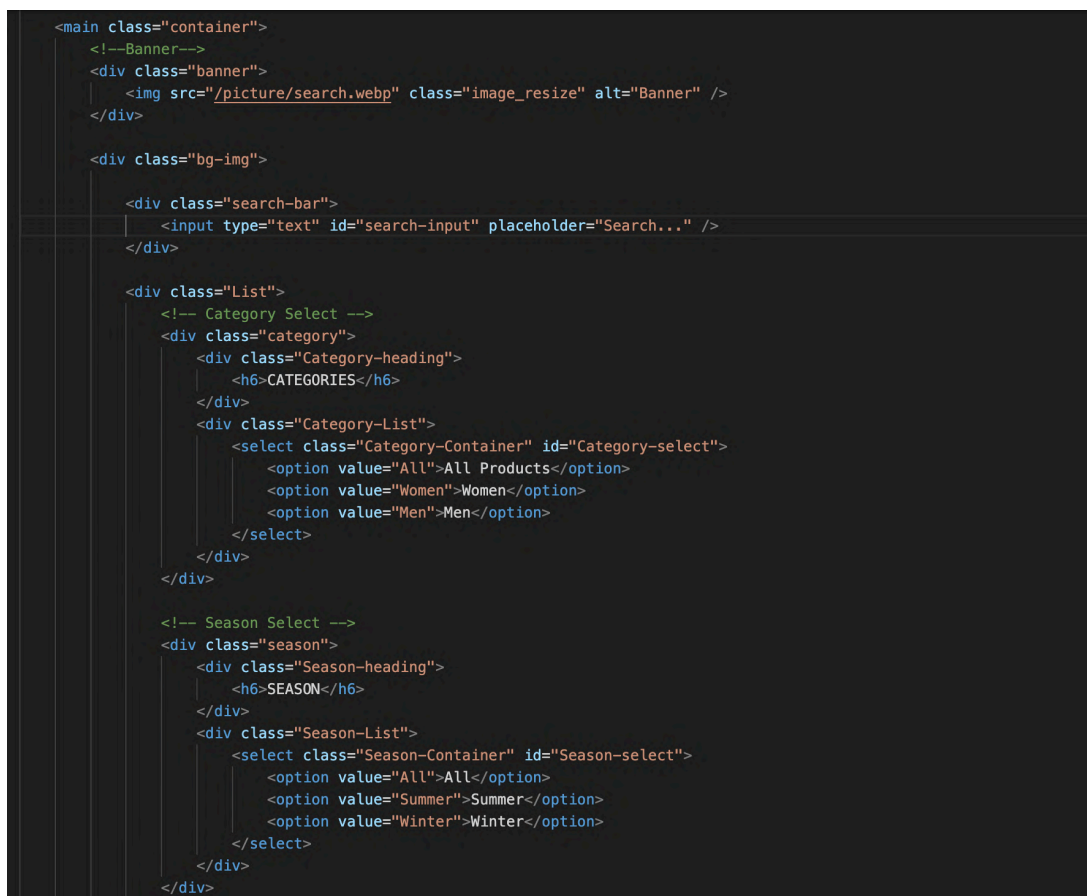
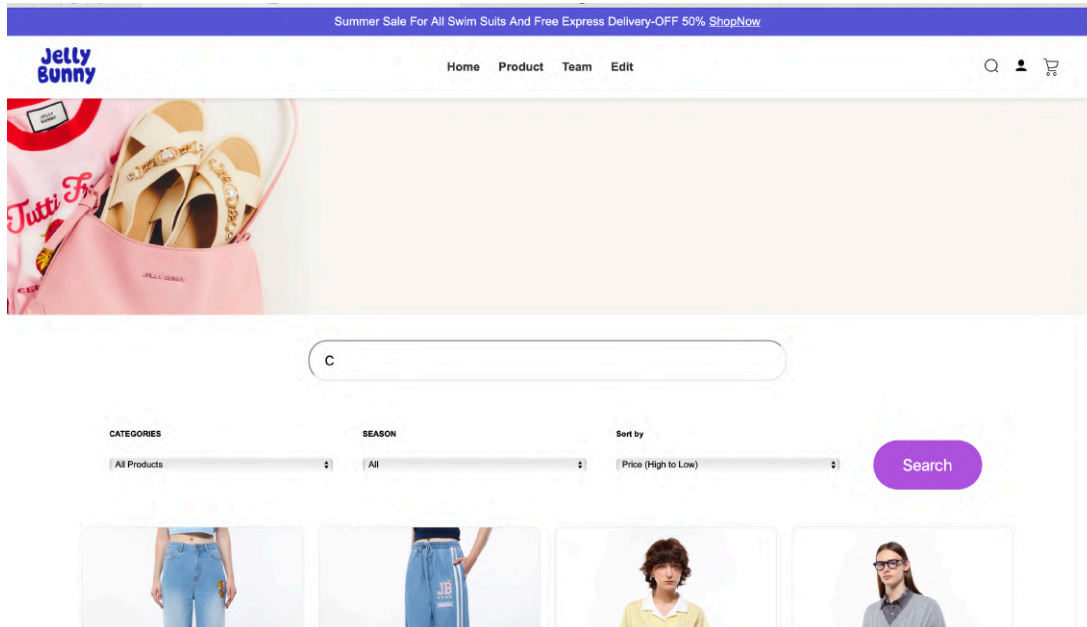
  fetch('http://localhost:8000/form-post', {
    method: "post",
    headers: {
      'Content-Type': 'application/json',
    },
    body: JSON.stringify({ username: tx_username, password: tx_password }),
  })
    .then(response => response.json())
    .then(data => {
      if (data.error == false) {
        setCookie("loginStatus", "true", 5);
        window.location.href = "http://localhost:3300/edit";
      } else {
        deleteCookie("loginStatus");
        alert(data.message);
      }
    })
}
</script>
```

This JavaScript code defines what happens when a user clicks the Sign In button:

1. Get Input Values: It reads values the user entered in the login form:
 - tx_username: grabs the text from the username fields.
 - tx_password: grabs the text from the password field.
2. Send a POST Request:
 - It sends a request to the server at <http://localhost:8000/form-post>.
 - It uses the POST method, meaning it sends data securely.
 - The data sent (body) is the username and password, converted into JSON format.
3. Handle the Server Response:
 - After sending the request, it waits for the server's reply.
 - If the server replies with error == false (meaning login success):
 - It sets a cookie named loginStatus with value “true”, lasting for 5 days.
 - It redirects the user to <http://localhost:3300/edit> (likely a dashboard or edit page).

- If the login fails (error == true):
 - It deletes the loginStatus cookie (if it exists).
 - It shows an alert with an error message (data.message).

- search.html



```

<!-- Sort Option -->
<div class="Sort-Container">
  <div class="Sort">
    <h6>Sort by</h6>
  </div>
  <div class="Sort-List">
    <select class="Sort-Container" id="Sort">
      <option value="nameAZ">Name (A-Z)</option>
      <option value="nameZA">Name (Z-A)</option>
      <option value="priceLowToHigh">Price (Low to High)</option>
      <option value="priceHighToLow">Price (High to Low)</option>
    </select>
  </div>
</div>

<div class="search-bar">
  <button id="search-btn" onclick="searchByName()">Search</button>
</div>

</div>

<!-- Product list container -->
<div class="product-list-container"></div>

```

Main Functionalities

- Search by Product Name, Category, and Season: Users can search for products by name, category, or season.
- Sort Results by Name or Price: Search results can be sorted by name or price.
- Display Results on Search: Relevant results will be displayed once the search button is clicked.

HTML Structure (User Interface)

1. Search Bar:
 - o The search bar allows the user to input a product name. It uses the `<input>` tag with `type="text"` and the `id="search-input"` to retrieve the value. The placeholder text says "Search...", indicating what users should enter in the field.
2. Category Filter:
 - o This section provides a dropdown to filter products by category. It includes options for All Products, Women, and Men. The `<select>` element has the `id="Category-select"`, and JavaScript will use this ID to retrieve the selected category.
3. Season Filter:
 - o This part allows users to filter products based on the season. The dropdown includes All, Summer, and Winter options, with the `id="Season-select"` used to reference this filter in JavaScript.
4. Sorting Option:
 - o Users can choose how to sort the search results, either by Name (A-Z), Name (Z-A), Price (Low to High), or Price (High to Low). The `<select>` tag is used to present the sorting options, and the `id="Sort"` will be used in JavaScript to apply the selected sorting method.
5. Search Button:
 - o A button with the `id="search-btn"` triggers the `searchByName()` JavaScript function, which performs the product search when clicked.
6. Product Results Container:

- o This is the section where the search results will be dynamically displayed. The `<div class="product-list-container">` is used to contain the list of products after the search has been performed.

```
<script>
function searchByName() {
  const productName = document.getElementById('search-input').value.trim();
  const category = document.getElementById('Category-select').value;
  const season = document.getElementById('Season-select').value;
  const sort = document.getElementById('Sort').value;

  if (productName || category !== 'All' || season !== 'All' || sort) {
    fetchProductData(productName, category, season, sort);
  }
}

window.addEventListener('load', function () {
  const productName = getUrlParam('productName') || '';
  const category = document.getElementById('Category-select').value;
  const season = document.getElementById('Season-select').value;
  const sort = document.getElementById('Sort').value;

  fetchProductData(productName, category, season, sort);
});

function getUrlParam(name) {
  const urlParams = new URLSearchParams(window.location.search);
  return urlParams.get(name);
}

function fetchProductData(productName, category, season, sortBy) {
  let url = 'http://localhost:8000/select';
  const queryParams = [];

  // Check each filter and add it to the query parameters
  if (productName) queryParams.push(`productName=${productName}`);
  if (category && category !== 'All') queryParams.push(`category=${category}`);
  if (season && season !== 'All') queryParams.push(`season=${season}`);
  if (sortBy) queryParams.push(`sortBy=${sortBy}`);

  // If any filters are applied, append them to the URL
  if (queryParams.length > 0) {
    url += '?' + queryParams.join('&');
  }

  console.log('Fetching from:', url);

  fetch(url)
    .then(response => response.json())
    .then(data => {
      const productListContainer = document.querySelector('.product-list-container');
      productListContainer.innerHTML = ''; // Clear the previous results

      let row = document.createElement('div');
      row.classList.add('Row');

      data.forEach(product => {
        const column = document.createElement('div');
        column.classList.add('column');

        const imageContainer = document.createElement('div');
        imageContainer.classList.add('image-container');
        const productImage = document.createElement('img');
        productImage.src = product.prod_pic1;
        productImage.alt = product.prod_name;
        imageContainer.appendChild(productImage);

        const productDetails = document.createElement('div');
        productDetails.classList.add('product-details');
        productDetails.innerHTML = `
          <h3 class="product-title">${product.prod_name}</h3>
          <p class="product-price">${product.prod_price}</p>
        `;

        column.appendChild(imageContainer);
        column.appendChild(productDetails);
      });
    });
}
```

```

        column.addEventListener('click', function () {
            window.location.href = `/detail-page?id=${product.prod_ID}`;
        });

        row.appendChild(column);

        if (row.children.length === 4) {
            productListContainer.appendChild(row);
            row = document.createElement('div');
            row.classList.add('Row');
        }
    });

    if (row.children.length > 0) {
        productListContainer.appendChild(row);
    }
})
.catch(error => console.error('Error:', error));
}

// Event listener to handle search action
document.getElementById('search-btn').addEventListener('click', function () {
    const productName = document.getElementById('search-input').value.trim();
    const category = document.getElementById('Category-select').value;
    const season = document.getElementById('Season-select').value;
    const sort = document.getElementById('Sort').value;

    fetchProductData(productName, category, season, sort);
});

function updateProductList() {
    const productName = getUrlParam('productName') || '';
    const category = document.getElementById('Category-select').value;
    const season = document.getElementById('Season-select').value;
    const sort = document.getElementById('Sort').value;

    fetchProductData(productName, category, season, sort);
}
</script>

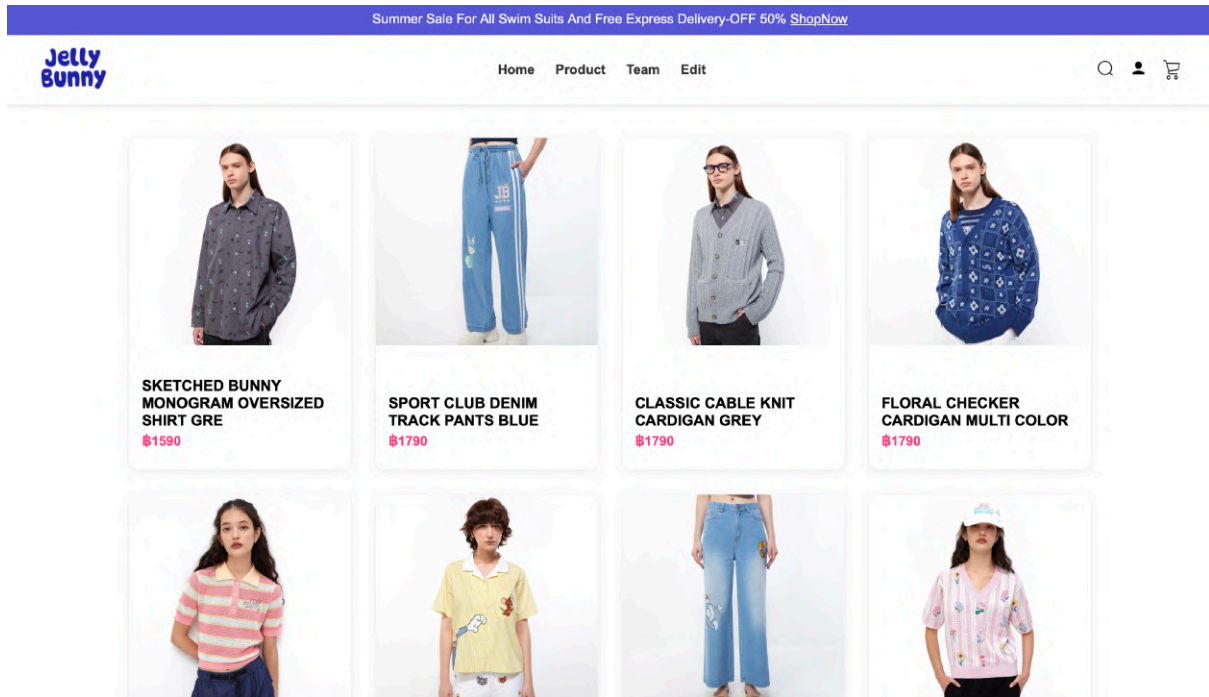
```

Script:

1. `searchByName()` Function:
 - o This function is triggered when the user searches for a product. It reads the input values from the search bar and dropdown filters (category, season, and sorting options).
 - o If the selected category or season is "All," or if a sort option is chosen, the function calls the `fetchProductData()` function to retrieve filtered data from the server.
2. `window.addEventListener('load', function() {...})`:
 - o This ensures that when the page loads, the current URL parameters (like product name, category, season, and sort options) are parsed and used to fetch the appropriate products.
3. `getUrlParam(name)` Function:
 - o This utility function extracts query parameters from the URL, which are used to initialize the filters on page load.
4. `fetchProductData()` Function:
 - o Constructs the request URL with the selected filter options and sends a GET request to the server.
 - o If there are filters applied, they are appended to the URL as query parameters (e.g., `productName`, `category`, `season`, and `sortBy`).
 - o Once the data is fetched, it dynamically updates the product list by creating HTML elements for each product and appending them to the page.
5. Product Rendering Logic:
 - o Inside `fetchProductData()`, after data is fetched, it creates HTML elements for each product such as image, name, and price, and appends them to a product container.
6. Event Listener for Product Click:

- o When a user clicks on a product (represented by column), an event listener redirects the user to a product detail page using the product's ID. This is done by setting the `window.location.href` to the product detail page with the corresponding `product.prod_ID`.
- 7. Dynamic Row Creation:
 - o Each product is placed inside a column, which is appended to a row. This ensures the layout is organized into rows with columns of products.
 - o A new row is created once four columns are added to ensure a clean layout.
- 8. `fetchProductData()` for Updating Products:
 - o The `fetchProductData()` function is called again when the search button is clicked to update the product list based on the current filter selections (product name, category, season, and sort).
- 9. Error Handling:
 - o A catch block is used to log any errors that occur during the fetch operation, ensuring that the application doesn't fail silently.

- Product.html



```
<!-- Removed all filters -->
<div class="product-list-container"></div>

<script>
window.addEventListener('load', function () {
    fetchProductData();
});

function fetchProductData() {
    const url = 'http://localhost:8000/select';

    fetch(url)
        .then(response => response.json())
        .then(data => {
            const productListContainer = document.querySelector('.product-list-container');
            productListContainer.innerHTML = '';

            let row = document.createElement('div');
            row.classList.add('Row');

            data.forEach(product => {
                const column = document.createElement('div');
                column.classList.add('column');

                const imageContainer = document.createElement('div');
                imageContainer.classList.add('image-container');
                const productImage = document.createElement('img');
                productImage.src = product.prod_pic1;
                productImage.alt = product.prod_name;
                imageContainer.appendChild(productImage);
            });
        });
}
```



```

const productDetails = document.createElement('div');
productDetails.classList.add('product-details');
productDetails.innerHTML = `
  <h3 class="product-title">${product.prod_name}</h3>
  <p class="product-price">฿${product.prod_price}</p>
`;

column.appendChild(imageContainer);
column.appendChild(productDetails);

column.addEventListener('click', function () {
  window.location.href = `/detail-page?id=${product.prod_ID}`;
});

row.appendChild(column);

if (row.children.length === 2) {
  productListContainer.appendChild(row);
  row = document.createElement('div');
  row.classList.add('Row');
}

if (row.children.length > 0) {
  productListContainer.appendChild(row);
}

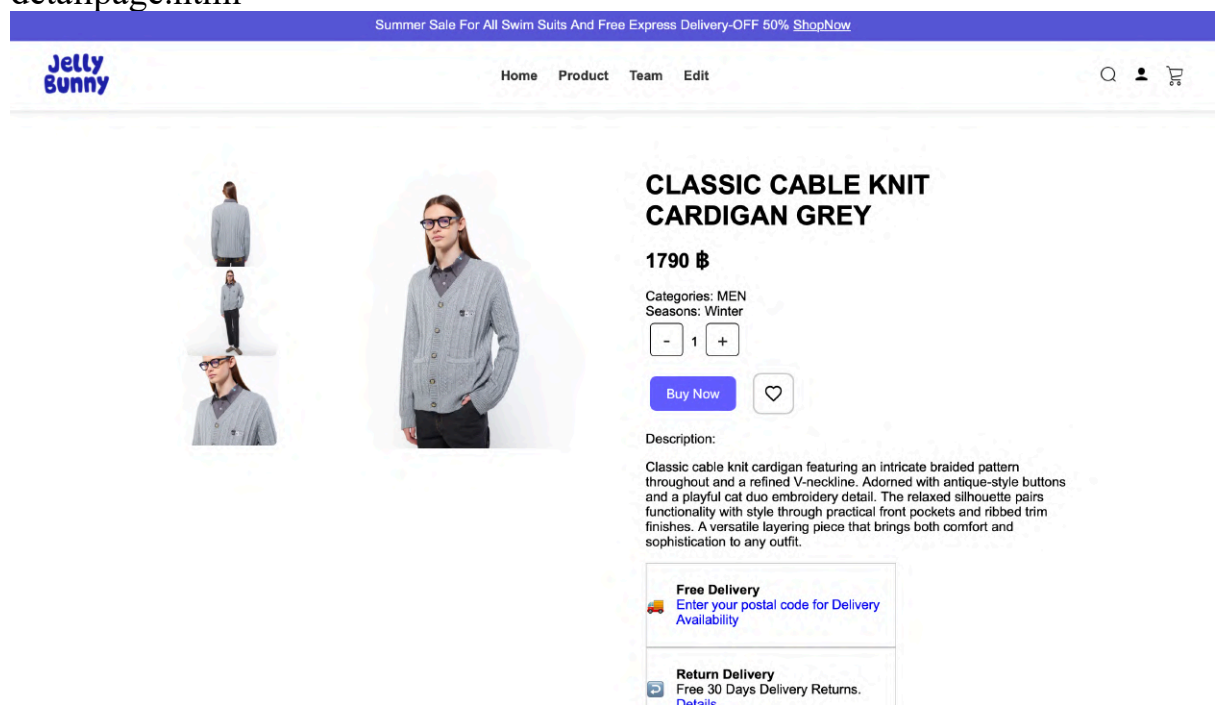
.catch(error => console.error('Error:', error));
}
</script>

```

1. Script: Start listening when page loads
 - o When the page finishes loading, it automatically calls the function `fetchProductData()`.
 - o `fetchProductData()` is responsible for getting the products from the server and showing them.
2. `fetchProductData()` function: This is the main function that loads and displays products.
 - o Setting up the URL
 - Tells JavaScript to fetch data from <http://localhost:8000/select>.
 - (select is probably an API endpoint from your backend that returns all products.)
 - o Making the HTTP request
 - `Fetch(url)`: Asks the server for data.
 - `Response.json()`: Converts the server's reply into a JavaScript object/array (probably an array of products).
 - Data will now hold an array of products.
3. Preparing the Container
 - o Selects the `<div>` where products will be displayed.
 - o Clears it in case there was old data from before.
4. Creating a Row: Creates a new Row `<div>` to hold two products side by side.
5. Looping through each product: For each product in the data array, it does the following.
6. For each product: Building the Product Card
 - o Create a Column: Makes a column `<div>` to hold one product.
 - o Create the Image Container: Create another `<div>` to hold the image separately.
 - o Add the Product Image
 - Creates an `<img>` element.
 - Sets the image source to the product's picture.
 - Sets the alt text for accessibility.
 - Adds the image into the image container.
 - o Create the Product Details
 - Creates a new `<div>` for name and price.
 - Sets the inner HTML:

- Product name inside an `<h3>`.
 - Product price inside a `<p>`.
 - o Assemble the Column: Adds the image and the product details into the column.
 - o Make the Column Clickable
 - When someone clicks the product, they are taken to the product's detail page.
 - o Add Column to Row: Adds the product's column to the current row.
 - o Check if Row is Full: If there are already 2 products in the row:
 - Adds the row into the page.
 - Start a new row for the next products.
7. After All Products: Check Remaining Products
- o If there are any products leftover (less than 2), still append the last row.
8. Error Handling: If something goes wrong (like server not responding), it shows an error in the browser console.

- detailpage.html




```

<main>

  <div class="product-container">

    <div class="product-gallery">

      <div class="details-pants-left" id="details-pants-left">

        <div> </div>

        <div> </div>

        <div> </div>

      </div>

      <div class="details-pants-right" id="details-pants-right">

      </div>

    </div>

    <div class="product-details">

      <h1 class="Productname"></h1>
      <p class="Status" id="Status">In Stock</p>

      <p>
        <h2 class="Price">1,790 ₺</h2>
      </p>

      <div class="option">
        Categories:
        <h4 id="Category">Category: </h4>
      </div>

      <div class="option">
        Seasons:
        <h4 id="Seasons">Seasons:</h4>
      </div>

      <div class="counter">
        <button onclick="decrement()">-</button>
        <span id="count">1</span>
        <button onclick="increment()">+</button>
      </div>

      <div class="button-container">
        <button class="buy-now">Buy Now</button>

        <button class="favorite-btn" onclick="toggleFavorite()">
          <svg viewBox="0 0 24 24">
            <path
              d="M12 21.35l-1.45-1.32C5.4 15.36 2 12.28 2 8.5 2 5.42 4.42 3 7.5 3c1.74 0 3.41.81 4.5 2.09C13.09 3
            </path>
          </svg>
        </button>
      </div>

    </div>
  </div>

```

```

<script>
  let count = 1;

  function increment() {
    count++;
    updateCount();
  }

  function decrement() {
    if (count > 1) {
      count--;
      updateCount();
    }
  }

  function updateCount() {
    document.getElementById('count').textContent = count;
  }

  function toggleFavorite() {
    document.querySelector('.favorite-btn').classList.toggle('active');
  }
}
</script>
<div>
  <p class="description">Description:</p>
  <p class="Description" id="Description">Blend sporty and sweet in these relaxed-fit denim track pants featuring playful JB varsity patches and side strips.</p>
</div>

<div class="section">
  <div class="icon">🚚</div>
  <div>
    <strong>Free Delivery</strong><br>
    <a href="#">Enter your postal code for Delivery Availability</a>
  </div>
</div>

<div class="section">
  <div class="icon">🔄</div>
  <div>
    <strong>Return Delivery</strong><br>
    Free 30 Days Delivery Returns. <a href="#">Details</a>
  </div>
</div>

```

1. Main Container

- o Everything (gallery + details + description) is wrapped inside this <div>.
- o Main structure of the product page.

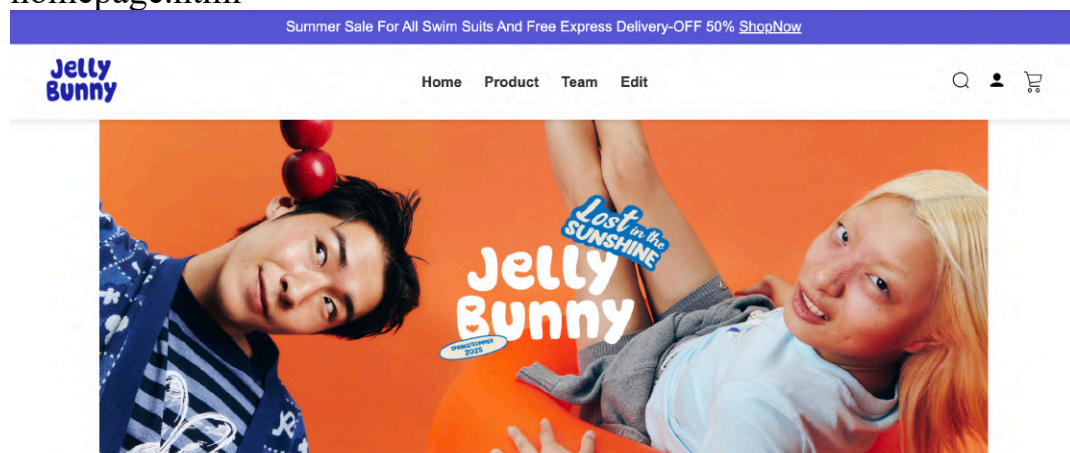
2. Product Gallery Section

- o A section specifically to show product images.
- o It's divided into two sides: Left (thumbnails) and Right (main image).
- o Left Side (Small images / thumbnails)
 - This part shows multiple smaller product images.
 - Inside, there are three <div> s, each containing an .
 - Purpose: Usually, when users click a thumbnail, it updates the main image (through your current code does not yet handle the click).
- o Right Side (Main big image)
 - This part shows the main big product image.
 - Initially loaded with a default image.
 - (In many shops, clicking a left image would change this big image: could be added later.)

3. Product Details Section: This parts shows product information like: Name, Stock status, Price, Category, Season, Description, Shipping/Return policies, Quantity selector, Buy/favorite buttons.

- o Basic Product Info
 - Name (currently empty).
 - Stock status (“In Stock” shown).
 - Price (1790 baht).
- o Category and Seasons
 - Shows which category (ex: “Pants”, “Shoes”, etc.) the product belongs to.
 - Shows season (ex: “Summer Collection”).
 - (h is used instead of h1-h6, but semantically it’s wrong: should use or <p>, because h is not a valid HTML tag.)
- o Quantity Counter
 - User can choose how many items to buy.
 - Buttons increase or decrease the number.
 - Start at 1.
- o Action Buttons
 - Buy Now button (no functionality linked yet).
 - Favorite button (adds/removes “favorite” status).
 - When clicked, it adds or removes the active CSS class.
 - (Normally, it would change color or look “selected” if favorited.)
- o Product Description
 - Two <p> tags:
 - Title: “Description:”
 - Actual description text of the product.
- o Delivery & Return Info
 - Two small sections with icons:
 - Free Delivery (users can check postal code).
 - Free Returns (within 30 days).

- homepage.html

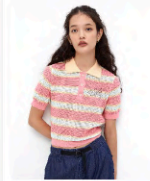


This Month

Best Selling Products



WAVE KNIT SHORT SLEEVE CARDIGAN
MULTI COLOR
฿1390



PASTEL STRIPE POLO KNIT TOP
MULTI COLOR
฿1390



TOM AND JERRY CHASE SCENE STRIPE BOWLING SHIRT
YELLOW
฿1790



GARDEN STRIPE V-NECK TOP
MULTI COLOR
฿1590



HAPPY 85TH BIRTHDAY TOM AND JERRY CROP TEE
GREY
฿1090

[View All Products](#)

Featured

New Arrival



TM & © Turner Ent. Co. (s25)



TOM AND JERRY CHASE SCENE STRIPE BOWLING SHIRT
YELLOW
฿1790



TOM AND JERRY CHASE SCENE DENIM JEANS
BLUE
฿2190



TOM AND JERRY DENIM JEANS
WHITE
฿2190



TOM AND JERRY SWEET ICE-CREAM KNIT PULLOVER
PINK
฿1790



HAPPY 85TH BIRTHDAY TOM AND JERRY CROP TEE
GREY
฿1090

[View All Products](#)



FREE AND FAST DELIVERY

Free delivery for all orders over ฿140



24/7 CUSTOMER SERVICE

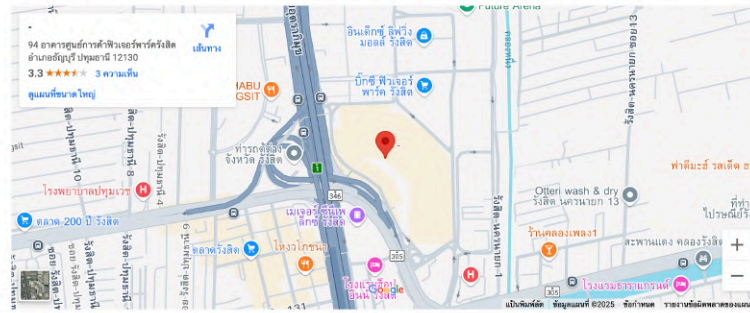
Friendly 24/7 customer support



MONEY BACK GUARANTEE

We return money within 30 days

Main branch of Jelly Bunny



Customer Support

FAQs
Store Location
Team

About Us

Terms of Conditions
Privacy Policy

Newsletter

Sign up to receive offers:

Enter your email

© Copyright JellyBunny, all rights reserved.

```
<!-- Main Content-->
<main class="main-content">

  <!--Banner-->
  <section class="banner">
    
  </section>

  <!--Best Selling Products-->
  <section class="best-selling">
    <h5 class="section-label">This Month</h5>
    <h2>Best Selling Products</h2>

    <div class="best-selling-grid">

    </div>

    <div class="view-products-wrapper">
      <button class="view-products-button" onclick="window.location.href='/product'">
        View All Products
      </button>
    </div>
  </section>
```



```

<!--New Arrival Section-->
<section class="new-arrivals">
  <h5 class="section-label">Featured</h5>
  <h2>New Arrival</h2>
  <div class="banner">
    
  </div>

  <div class="new-arrivals-container">

    <div class="new-arrivals-grid">
      <article class="product-card1">
        <div class="product-image-container">
          <img class="product-image">
          <div class="product-badge"></div>
        </div>
        <h3 class="product-name"></h3>
        <div class="product-price"></div>
        <div class="see-more-button"></div>
      </article>
    </div>
  </div>
  <div class="view-products-wrapper">
    <button class="view-products-button" onclick="window.location.href='/product'">
      View All Products
    </button>
  </div>
</section>

<!-- Features -->
<section class="features">
  <div class="features-text">
    <div class="icon">
      
    <div class="icon">
      
    </div>
    <h3>24/7 CUSTOMER SERVICE</h3>
    <p>Friendly 24/7 customer support</p>
  </div>
  <div class="features-text">
    <div class="icon">
      
    </div>
    <h3>MONEY BACK GUARANTEE</h3>
    <p>We return money within 30 days</p>
  </div>
</section>

<h2>Main branch of Jelly Bunny</h2>
<iframe width="100%" height="400" style="border:0" loading="lazy" allowfullscreen
  src="https://www.google.com/maps/embed/v1/place?key=AIzaSyCv0-1cJxCdo2jXibSkjQtN-DlvVIPVoZo&q=Jelly+Bunny+Zpell+R
</iframe>

</main>

```

```

<script>

  fetch("http://localhost:8000/best-selling")
    .then(response => response.json())
    .then(data => {
      const productGrid = document.querySelector('.best-selling-grid');
      productGrid.innerHTML = '';

      data.forEach(product => {
        const productCard = document.createElement('article');
        productCard.classList.add('product-card2');
        productCard.style.cursor = 'pointer';

        const productImageContainer = document.createElement('div');
        productImageContainer.classList.add('product-image-container');

        const productImage = document.createElement('img');
        productImage.classList.add('product-image');
        productImage.src = product.prod_pic1;

        const productBadge = document.createElement('div');
        productBadge.classList.add('product-badge');
        productBadge.textContent = product.status;

        productImageContainer.appendChild(productImage);
        productImageContainer.appendChild(productBadge);

        const productName = document.createElement('h3');
        productName.classList.add('product-name');
        productName.textContent = product.prod_name;

        const productPrice = document.createElement('div');
        productPrice.classList.add('product-price');
        productPrice.textContent = `฿${product.prod_price}`;

        productCard.appendChild(productImageContainer);
        productCard.appendChild(productName);
        productCard.appendChild(productPrice);

        productCard.addEventListener('click', () => {
          window.location.href = `/detail-page?id=${product.prod_ID}`;
        });

        productGrid.appendChild(productCard);
      });
    })
    .catch(error => console.error('Error fetching best-selling products:', error));

```



```

fetch("http://localhost:8000/new-products")
  .then(response => response.json())
  .then(data => {
    const productGrid = document.querySelector('.new-arrivals-grid');
    productGrid.innerHTML = '';

    data.forEach(product => {
      const productCard = document.createElement('article');
      productCard.classList.add('product-card1');
      productCard.style.cursor = 'pointer';

      const productImageContainer = document.createElement('div');
      productImageContainer.classList.add('product-image-container');

      const productImage = document.createElement('img');
      productImage.classList.add('product-image');
      productImage.src = product.prod_pic1;

      const productBadge = document.createElement('div');
      productBadge.classList.add('product-badge');
      productBadge.textContent = product.status;

      productImageContainer.appendChild(productImage);
      productImageContainer.appendChild(productBadge);

      const productName = document.createElement('h3');
      productName.classList.add('product-name');
      productName.textContent = product.prod_name;

      const productPrice = document.createElement('div');
      productPrice.classList.add('product-price');
      productPrice.textContent = `฿${product.prod_price}`;

      productCard.appendChild(productImageContainer);
      productCard.appendChild(productName);
      productCard.appendChild(productPrice);

      productCard.addEventListener('click', () => {
        window.location.href = `/detail-page?id=${product.prod_ID}`;
      });

      productGrid.appendChild(productCard);
    });
  })
  .catch(error => console.error('Error fetching new products:', error));

```

1. Main Layout

- o All content is wrapped inside a <main> tag.
- o This is the main area of the page.

2. Banner Section

- o Displays a large image banner at the top.
- o (Usually used for promotions, sales, etc.)

3. Best-Selling Products Section

- o Title: “This Month” + “Best Selling Products”.
- o Best Selling Grid:
 - Initially empty.
 - It will be filled later with products dynamically using JavaScript.
- o Button to view all products:When clicked, redirects the user to the /product page.

4. New Arrivals Section

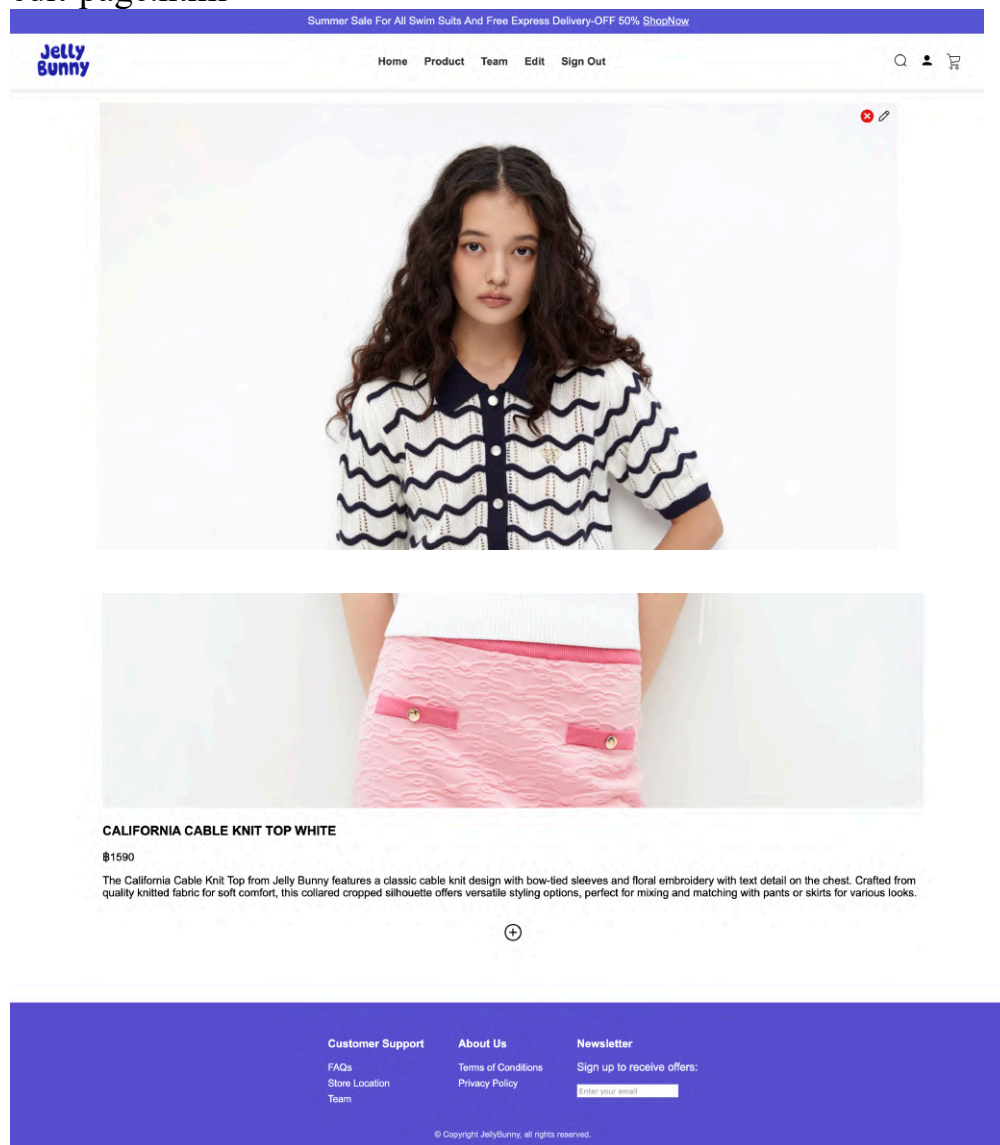
- o Title: “Featured” + “New Arrival”.
- o Another banner image shown.
- o A small product card placeholder is created inside.

5. Features Section: Each features has:

- o An icon image (loaded from online).

- o A title (like “FREE AND FAST DELIVERY”).
- o A small description (like “Free delivery for all orders over \$140”).
- 6. Store Location (Map)
 - o Shows a Google Map iframe.
 - o Pinpoints the main store location of Jelly Bunny.
- 7. Dynamic Product Fetching (JavaScript)
 - o Fetches best-selling product data from your backend API (http://localhost:8000/best-selling).

- edit-page.html



```

<main>
  <div class="product-list-container"></div>

  <script>

    window.addEventListener('load', function () {
      let loginStatus = getCookie("loginStatus");
      if (loginStatus == 'true') {
        document.getElementById("btSignOut").style.display = "block";
        fetchProductData();
      } else {
        document.getElementById("btSignOut").style.display = "none";
        window.location.href = "http://localhost:3300/sign-in";
      }
    });

    function SignOut() {
      deleteCookie("loginStatus");
      window.location.href = "http://localhost:3300/homepage";
    }

    function fetchProductData() {
      const url = 'http://localhost:8000/select';

      fetch(url)
        .then(response => response.json())
        .then(data => {
          const productListContainer = document.querySelector('.product-list-container');
          productListContainer.innerHTML = '';

          let row = document.createElement('div');
          row.classList.add('Row');

```

This document describes the functionality implemented in the edit-page.html file, focusing on product display, user authentication handling, product editing, and deletion features.

1. Page Load Behavior

When the page loads, the script checks the user's login status via a cookie named loginStatus:

- If the user is logged in (loginStatus == "true"):
 - The **Sign Out** button is displayed.
 - The product data is fetched from the server and displayed dynamically.
- If the user is not logged in:
 - The **Sign Out** button is hidden.
 - The user is redirected to the login page (<http://localhost:3300/sign-in>).

2. User Sign Out

The SignOut() function handles user sign-out by:

- Deleting the loginStatus cookie.
- Redirecting the user to the homepage (<http://localhost:3300/homepage>).

```

data.forEach(product => {
  const column = document.createElement('div');
  column.classList.add('column');

  const imageContainer = document.createElement('div');
  imageContainer.classList.add('image-container');
  const productImage = document.createElement('img');
  productImage.src = product.prod_pic1;
  productImage.alt = product.prod_name;
  imageContainer.appendChild(productImage);

  const productDetails = document.createElement('div');
  productDetails.classList.add('product-details');
  productDetails.innerHTML = `
    <h3 class="product-title">${product.prod_name}</h3>
    <p class="product-price">฿${product.prod_price}</p>
    <p class="product-description">${product.prod_description}</p>
  `;

  // Create the tool (small images at the top right corner)
  const toolContainer = document.createElement('div');
  toolContainer.classList.add('tool-container');

  // Create the Delete icon
  const toolImage1 = document.createElement('img');
  toolImage1.src = '/picture/delete.png'; // Delete icon
  toolImage1.alt = 'Delete';
  toolImage1.classList.add('tool-image');
  toolImage1.onclick = function () { openPopup(product.prod_ID); }; // Open popup for deletion

  // Create the Pencil (Edit) icon
  const toolImage2 = document.createElement('img');
  toolImage2.src = '/picture/pencil.png'; // Edit icon
  toolImage2.alt = 'Edit';
  toolImage2.classList.add('tool-image');
  toolImage2.onclick = function () {
    // Change the page to the edit page with product ID as a query parameter
    window.location.href = `/edit-product?id=${product.prod_ID}`; // Redirect to edit page
  };

  toolContainer.appendChild(toolImage1);
  toolContainer.appendChild(toolImage2);

  // Append the tool container to the image container
  imageContainer.appendChild(toolContainer);

  column.appendChild(imageContainer);
  column.appendChild(productDetails);

  column.addEventListener('click', function () {

  });

  row.appendChild(column);

  if (row.children.length === 2) {
    productListContainer.appendChild(row);
    row = document.createElement('div');
    row.classList.add('Row');
  }
});

```

```

  if (row.children.length > 0) {
    productListContainer.appendChild(row);
  }
})
.catch(error => console.error('Error:', error));
}

```

```

// Popup functions
function openPopup(productId) {
    document.getElementById("popup").style.display = "block";
    document.getElementById("overlay").style.display = "block";

    window.productToDelete = productId; // Store the product to delete
}

function closePopup() {
    document.getElementById("popup").style.display = "none";
    document.getElementById("overlay").style.display = "none";
}

function confirmDelete() {
    const sku = window.productToDelete;
    fetch(`http://localhost:8000/delete_product?sku=${sku}`, {
        method: 'DELETE'
    })
    .then(response => {
        if (response.ok) {
            console.log(`Product with SKU ${sku} deleted successfully.`);
            // After successful deletion, fetch updated product data
            //const selectedCategory = document.getElementById('Category-select').value;
            //const selectedSortingOption = document.getElementById('Sort').value;
            //fetchProductData(selectedCategory, selectedSortingOption);
            fetchProductData();
        } else {
            console.error(`Failed to delete product with SKU ${sku}.`);
        }
        // Hide the delete confirmation popup regardless of the response
        closePopup();
    })
    .catch(error => {
        console.error('Error deleting product:', error);
        // Hide the delete confirmation popup in case of an error
        closePopup();
    });

    console.log("Item with ID " + window.productToDelete + " will be deleted.");
    // Here you can send a request to the server to delete the product
}

<!-- Overlay background -->
<div id="overlay" class="overlay" onclick="closePopup()"></div>

<!-- Popup box -->
<div id="popup" class="popup">
    <p>Do you want to delete this item?</p>
    <button class="cancel" onclick="closePopup()">CANCEL</button>
    <button class="confirm" onclick="confirmDelete()">CONFIRM</button>
</div>
<a href="/add">
    <button class="add">
        
    </button>
</a>

```

3. Fetching and Displaying Products

The `fetchProductData()` function retrieves product data from the server endpoint (`http://localhost:8000/select`) using a `fetch()` request. Upon successful retrieval:

- Each product is displayed inside a dynamically created `<div>` element.
- Product information displayed includes:
 - o Product image
 - o Product name

- o Product price
- o Product description

For each product, two control icons are provided:

- **Delete Icon:** Opens a popup for delete confirmation.
- **Edit Icon:** Redirects the user to the edit page, passing the product ID as a query parameter (/edit-product?id={productID}).

4. Deleting a Product

The deletion process is handled through a confirmation popup:

- Clicking the delete icon triggers `openPopup(productId)`, displaying a popup and storing the selected product ID globally.
- Confirming deletion (`confirmDelete()`) sends a DELETE HTTP request to: http://localhost:8000/delete_product?sku={sku}
- Upon successful deletion:
 - o A success message is logged.
 - o The product list is refreshed by calling `fetchProductData()` again.
- Whether deletion succeeds or fails, the popup is closed after the process.

The popup structure consists of:

- A semi-transparent background (overlay) that can also close the popup.
- Two buttons inside the popup: CANCEL and CONFIRM.

4. Adding a New Product

At the bottom of the page, an Add (+) button is provided:

- Click the button redirects the user to /add (likely `add.html`), where they can add a new product.

- edit.html

Summer Sale For All Swim Suits And Free Express Delivery-OFF 50% ShopNow

Jelly Bunny

HomeProductTeamEdit

Edit Product

ID:

PRD59AMJYWU1E

Product Name:

WAVE KNIT SHORT SLEEVE CARDIGAN MULTI COLOR RAINBOW

Product Status:

NEW

Price:

1390

Categories: ☒ WOMEN ☐ MEN

Seasons: ☐ Winter ☒ Summer

Description:

The Wave Knit Cardigan from Jelly Bunny features a classic design with chic two-tone wave pattern, decorated with pearl buttons and metallic logo detail. Ribbed collar, cuffs, and hem add refined finishing touches. Crafted from quality knitted fabric for soft comfort, this cropped length cardigan is perfect for mix-and-match styling.

Quantity:

8

Image 1 URL:

https://jellybunny.com/cdn/shop/files/B25SATL017_MLT000_1_34a9797b-cb1b-428b-8872-07b68b10413e

Image 2 URL:

https://jellybunny.com/cdn/shop/files/B25SATL017_MLT000_2_91c240be-6b74-436e-8899-e983e50c431

Image 3 URL:

https://jellybunny.com/cdn/shop/files/B25SATL017_MLT000_3_39e78f7a-26b6-4630-b3da-19e6ab654d7

Image 4 URL:

https://jellybunny.com/cdn/shop/files/B25SATL017_MLT000_4_1f868b29-d09c-41f3-a808-13d71d4c87fa

CANCEL

SAVE

Customer Support

FAQs

Store Location

Team

About Us

Terms of Conditions

Privacy Policy

Newsletter

Sign up to receive offers:

Enter your email


```

<div class="Add-product">
  <div class="Add-product-header">
    <h1>Edit Product</h1>
  </div>

  <div class="Add-product-container">
    <div class="product-details">
      <fieldset>
        <form id="editProductForm">
          <div class="Label">
            <label for="txtID">ID:</label>
            <input type="text" id="txtID" name="txtID" required disabled>

            <label for="txtProductname">Product Name:</label>
            <input type="text" id="txtProductname" name="txtProductname" required>

            <label for="txtStatus">Product Status:</label>
            <input type="text" id="txtStatus" name="txtStatus" required>
          </div>

          <div class="Label">
            <label for="txtPrice">Price:</label>
            <input type="number" id="txtPrice" name="txtPrice" required>
          </div>

          <div class="option">
            Categories:
            <input type="radio" name="product-categories" value="WOMEN" required> WOMEN
            <input type="radio" name="product-categories" value="MEN"> MEN
          </div>

          <div class="option">
            Seasons:
            <input type="radio" name="product-season" value="Winter" required> Winter
            <input type="radio" name="product-season" value="Summer"> Summer
          </div>

          <div class="Label">
            <label for="txtDescription">Description:</label>
            <textarea id="txtDescription" name="txtDescription" rows="9" cols="10" required></textarea>
          </div>

          <div class="Label">
            <label for="txtQuantity">Quantity:</label>
            <input type="number" id="txtQuantity" name="txtQuantity" required>
          </div>

          <div class="Label">
            <label for="txtImage1">Image 1 URL:</label>
            <input type="url" id="txtImage1" name="txtImage1" required>
          </div>
          <div class="Label">
            <label for="txtImage2">Image 2 URL:</label>
            <input type="url" id="txtImage2" name="txtImage2">
          </div>
          <div class="Label">
            <label for="txtImage3">Image 3 URL:</label>
            <input type="url" id="txtImage3" name="txtImage3">
          </div>
          <div class="Label">
            <label for="txtImage4">Image 4 URL:</label>
            <input type="url" id="txtImage4" name="txtImage4">
          </div>

          <div class="button-container">
            <button type="reset" onclick="goToProductManagement()">CANCEL</button>
            <button type="submit" id="saveBtn">SAVE</button>
          </div>
        </form>
      </fieldset>
    </div>
  </div>
</div>

```

1. HTML Structure

This main part of the UI is a form inside a container:

- Header: Displays the title "Edit Product".
- Form Fields:
 - o ID: Product ID field (disabled to prevent editing).

- o Product Name: Editable text input.
- o Product Status: Editable text input.
- o Price: Editable number input.
- o Categories: Radio buttons (WOMEN or MEN).
- o Seasons: Radio buttons (Winter or Summer).
- o Description: Textarea input.
- o Quantity: Editable number input.
- o Image URLs: Inputs for up to 4 images.
- Buttons:
 - o CANCEL: Resets the form and calls goToProductManagement() to navigate elsewhere.
 - o SAVE: Submits the form and triggers saving the edited data.

```
<script>

function fetchProductDetails() {

  const urlParams = new URLSearchParams(window.location.search);
  const id = urlParams.get('id');

  fetch(`http://localhost:8000/product_detail?prod_ID=${id}`)
    .then(response => response.json())
    .then(product => {

      // data.forEach(product => {

      document.getElementById('txtID').value = product.prod_ID;
      document.getElementById('txtProductname').value = product.prod_name;
      document.getElementById('txtStatus').value = product.prod_status;
      document.getElementById('txtPrice').value = product.prod_price;
      document.getElementById('txtDescription').value = product.prod_description;
      document.getElementById('txtQuantity').value = product.prod_quantity;
      document.getElementById('txtImage1').value = product.prod_pic1;
      document.getElementById('txtImage2').value = product.prod_pic2;
      document.getElementById('txtImage3').value = product.prod_pic3;
      document.getElementById('txtImage4').value = product.prod_pic4;

      const categories = product.prod_category.toUpperCase();
      const checkboxes_categories = document.getElementsByName('product-categories');
      checkboxes_categories.forEach(checkbox => {
        if (checkbox.value.toUpperCase() == categories) {
          checkbox.checked = true;
        }
      });
    });
}
```

```

        const seasons = product.prod_seasons.toUpperCase();
        const checkboxes_season = document.getElementsByName('product-season');
        checkboxes_season.forEach(checkbox => {
            if (checkbox.value.toUpperCase() == seasons) {
                checkbox.checked = true;
            }
        });
    });
}

.catch(error => console.error('Error fetching product details:', error));
}

// Handle form submission
document.getElementById('editProductForm').addEventListener('submit', saveProduct);

function saveProduct(event) {
    event.preventDefault(); // Prevent form submission

    // Get product data from form fields
    const prod_ID = document.getElementById('txtID').value;
    const tx_prod_name = document.getElementById('txtProductname').value;
    const tx_prod_status = document.getElementById('txtStatus').value;
    const tx_prod_price = document.getElementById('txtPrice').value;
    const tx_prod_category = getSelectedCategory();
    const tx_prod_seasons = getSelectedSeason();
    const tx_prod_description = document.getElementById('txtDescription').value;
    const tx_prod_quantity = document.getElementById('txtQuantity').value;
    const tx_prod_pic1 = document.getElementById('txtImage1').value;
    const tx_prod_pic2 = document.getElementById('txtImage2').value;
    const tx_prod_pic3 = document.getElementById('txtImage3').value;
    const tx_prod_pic4 = document.getElementById('txtImage4').value;

    // Create product object
    const editProduct = {
        prod_name: tx_prod_name,
        prod_status: tx_prod_status,
        prod_pic1: tx_prod_pic1,
        prod_pic2: tx_prod_pic2,
        prod_pic3: tx_prod_pic3,
        prod_pic4: tx_prod_pic4,
        prod_category: getSelectedCategory(),
        prod_seasons: getSelectedSeason(),
        prod_description: tx_prod_description,
        prod_price: tx_prod_price,
        prod_quantity: tx_prod_quantity
    };

    // Send data to the server using fetch
    fetch('http://localhost:8000/update-product?prod_ID=${prod_ID}', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
        },
        body: JSON.stringify({ Product: editProduct })
    })
    .then(response => {
        if (!response.ok) {
            // throw new Error('Network response was not ok');
            alert('Error updating product into the database');
        }
        if (response.ok) {
            alert('Product has been updated successfully.');
```

<http://localhost:3300/edit>

```

            //return response.json(); // Parse JSON response
        }
    })
}

```

```

        .then(data => {
            console.log(data); // Log the response from the server
            //alert(data.alertMessage); // Display the alert message
        })
        .catch(error => {
            console.log('Error:', error);
        });
    }

    // Helper function to get selected category
    function getSelectedCategory() {
        const categories = document.getElementsByName('product-categories');
        for (let i = 0; i < categories.length; i++) {
            if (categories[i].checked) {
                return categories[i].value;
            }
        }
        return null; // Return null if no category is selected
    }

    // Helper function to get selected season
    function getSelectedSeason() {
        const seasons = document.getElementsByName('product-season');
        for (let i = 0; i < seasons.length; i++) {
            if (seasons[i].checked) {
                return seasons[i].value;
            }
        }
        return null; // Return null if no season is selected
    }

    function setSelectedCategory(vValue) {
        const categories = document.getElementsByName('product-categories');
        for (let i = 0; i < categories.length; i++) {
            if (categories[i].value == vValue) {
                return categories[i].checked = true;
            }
        }
        return null; // Return null if no category is selected
    }

    // Helper function to get selected season
    function setSelectedSeason(vValue) {
        const seasons = document.getElementsByName('product-season');
        for (let i = 0; i < seasons.length; i++) {
            if (seasons[i].value == vValue) {
                return seasons[i].checked = true;
            }
        }
        return null; // Return null if no season is selected
    }

    // Redirect function
    function goToProductManagement() {
        window.location.href = 'http://localhost:3300/edit';
    }

    window.addEventListener('load', function () {
        let loginStatus = getCookie("loginStatus");
        if (loginStatus == 'true') {
            fetchProductDetails();
        } else {
            window.location.href = "http://localhost:3300/sign-in";
        }
    });

```

2. JavaScript

- Fetching Product Details (fetchProductDetails())

- Purpose: When the page loads, this function fetches existing product details based on the id in the URL query string.
- Process:
 - o Extracts the product ID from the URL (window.location.search).
 - o Sends a GET request to `http://localhost:8000/product_detail?prod_ID={id}`.
 - o On successful response:
 - Fills the form inputs with the product's current data.
 - Selects the correct category and season radio buttons based on the product info.
- Handling Form Submission (saveProduct(event))
 - Purpose: Saves the edited product information back to the server.
 - Process:
 - o Prevents the default form submission.
 - o Gathers all values from form fields into an object (editProduct).
 - o Sends a POST request to: http://localhost:8000/update-product?prod_ID={prod_ID} with the updated product data in JSON format.
 - o If the server responds successfully:
 - Alerts the user of success.
 - Redirects the user back to the edit page list (`http://localhost:3300/edit`).
 - o If there is an error:
 - Displays an alert saying there was an error updating the product.
- add.html

Summer Sale For All Swim Suits And Free Express Delivery-OFF 50% ShopNow

Jelly Bunny Home Product Team Edit 🔍 👤 🛒

Add New Product

ID:

Product Name:

Product Status:

Price:

Categories: ☐ WOMEN ☐ MEN

Seasons: ☐ Winter ☐ Summer

Description:

Quantity:

Image 1 URL:

Image 2 URL:

Image 3 URL:

Image 4 URL:

CANCEL

SAVE

```

<div class="Add-product">
  <div class="Add-product-header">
    <h1>Add New Product</h1>
  </div>

  <div class="Add-product-container">
    <div class="product-details">
      <fieldset>
        <form id="addProductForm">
          <div class="Label">
            <label for="txtID">ID:</label>
            <input type="text" id="txtID" name="txtID" required>

            <label for="txtProductname">Product Name:</label>
            <input type="text" id="txtProductname" name="txtProductname" required>

            <label for="txtStatus">Product Status:</label>
            <input type="text" id="txtStatus" name="txtStatus" required>
          </div>

          <div class="Label">
            <label for="txtPrice">Price:</label>
            <input type="number" id="txtPrice" name="txtPrice" required>
          </div>

          <div class="option">
            Categories:
            <input type="radio" name="product-categories" value="WOMEN" required> WOMEN
            <input type="radio" name="product-categories" value="MEN"> MEN
          </div>

          <div class="option">
            Seasons:
            <input type="radio" name="product-season" value="Winter" required> Winter
            <input type="radio" name="product-season" value="Summer" required> Summer
          </div>
        </form>
      </fieldset>
    </div>
  </div>
</div>

```



```

<div class="Label">
  <label for="txtDescription">Description:</label>
  <textarea id="txtDescription" name="txtDescription" rows="9" cols="10" required></textarea>
</div>

<div class="Label">
  <label for="txtQuantity">Quantity:</label>
  <input type="number" id="txtQuantity" name="txtQuantity" required>
</div>

<div class="Label">
  <label for="txtImage1">Image 1 URL:</label>
  <input type="url" id="txtImage1" name="txtImage1" required>
</div>
<div class="Label">
  <label for="txtImage2">Image 2 URL:</label>
  <input type="url" id="txtImage2" name="txtImage2">
</div>
<div class="Label">
  <label for="txtImage3">Image 3 URL:</label>
  <input type="url" id="txtImage3" name="txtImage3">
</div>
<div class="Label">
  <label for="txtImage4">Image 4 URL:</label>
  <input type="url" id="txtImage4" name="txtImage4">
</div>

<div class="button-container">
  <button type="reset" onclick="goToProductManagement()">CANCEL</button>
  <button type="submit" id="saveBtn">SAVE</button>
</div>
</form>
</fieldset>

```

The **Add New Product** page is designed for administrators to add new products to the catalog. The page includes the following components:

1. Container:
 - o `<div class="Add-product">`: A container for the entire Add New Product page.
2. Page Title:
 - o `<h1>Add New Product</h1>`: The title or header of the page.
3. Product Information Form:
 - o `<form id="addProductForm">`: A form where the admin fills in the details of the new product:
 - Product ID: Text input field for entering the product's unique ID.
 - Product Name: Text input field for entering the name of the product.
 - Product Status: Text input field (e.g., "Available", "Out of Stock") for the status of the product.
 - Price: Number input field for entering the product's price.
 - Category: Radio buttons to choose the category: WOMEN or MEN.
 - Season: Radio buttons to select the season: Winter or Summer.
 - Description: A textarea to provide a detailed description of the product.
 - Quantity: Number input field for entering the quantity of the product in stock.
 - Image URLs: Up to 4 input fields to provide image URLs for the product.
4. Buttons:
 - o CANCEL button: Calls the `goToProductManagement()` function and redirects the admin to the product management page (/edit).
 - o SAVE button: Submits the form to save the new product to the catalog.


```

<script>
  // Handle form submission
  window.addEventListener('load', function () {
    let loginStatus = getCookie("loginStatus");
    if (loginStatus == 'true') {

    } else {

      window.location.href = "http://localhost:3300/sign-in";

    }

  });

  document.getElementById('addProductForm').addEventListener('submit', saveProduct);

  function saveProduct(event) {
    event.preventDefault(); // Prevent form submission

    // Get product data from form fields
    const prod_ID = document.getElementById('txtID').value;
    const prod_name = document.getElementById('txtProductname').value;
    const prod_status = document.getElementById('txtStatus').value;
    const prod_price = document.getElementById('txtPrice').value;
    const prod_category = getSelectedCategory();
    const prod_seasons = getSelectedSeason();
    const prod_description = document.getElementById('txtDescription').value;
    const prod_quantity = document.getElementById('txtQuantity').value;
    const prod_pic1 = document.getElementById('txtImage1').value;
    const prod_pic2 = document.getElementById('txtImage2').value;
    const prod_pic3 = document.getElementById('txtImage3').value;
    const prod_pic4 = document.getElementById('txtImage4').value;

    // Create product object
    const newProduct = {
      prod_ID,
      prod_name,
      prod_status,
      prod_pic1,
      prod_pic2,
      prod_pic3,
      prod_pic4,
      prod_category,
      prod_seasons,
      prod_description,
      prod_price,
      prod_quantity
    };

    // Send data to the server using fetch
    fetch('http://localhost:8000/add-product', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ Product: newProduct })
    })
    .then(response => {

      if (!response.ok) {
        // throw new Error('Network response was not ok');
        alert('Error inserting product into the database');
      }

      if (response.ok) {
        alert('New product has been created successfully. ');
        window.location.href = 'http://localhost:3300/product';
        //return response.json(); // Parse JSON response
      }
    })
  }

```

JavaScript

1. Form Submission Handling:

- o The code listens for the form submission on the "Add Product" page using `addEventListener('submit', saveProduct)`.

- o The saveProduct function is triggered when the form is submitted. It prevents the default form submission using event.preventDefault() to control the process manually.
- 2. Extracting Form Data:
 - o The values from each form field are retrieved using document.getElementById() and stored in variables such as prod_ID, prod_name, prod_status, etc.
 - o These values are taken from the input fields for Product ID, Name, Status, Price, Category, Season, Description, Quantity, and Image URLs.
- 3. Creating the Product Object:
 - o A new product object, newProduct, is created using the values gathered from the form fields.
 - o This object will later be sent to the server as part of the request payload.

```

    })
    .then(data => {
        console.log(data); // Log the response from the server
    })
    .catch(error => {
        console.log('Error:', error);
    });
}

// Helper function to get selected category
function getSelectedCategory() {
    const categories = document.getElementsByName('product-categories');
    for (let i = 0; i < categories.length; i++) {
        if (categories[i].checked) {
            return categories[i].value;
        }
    }
    return null; // Return null if no category is selected
}

// Helper function to get selected season
function getSelectedSeason() {
    const seasons = document.getElementsByName('product-season');
    for (let i = 0; i < seasons.length; i++) {
        if (seasons[i].checked) {
            return seasons[i].value;
        }
    }
    return null; // Return null if no season is selected
}

// Redirect function
function goToProductManagement() {
    window.location.href = 'http://localhost:3300/edit';
}
</script>

```

- 4. Sending Data to the Server:
 - o The fetch() function is used to send the data to the server. The method is POST, and the content type is set to application/json.
 - o The newProduct object is serialized into JSON using JSON.stringify() and sent as the request body.
- 5. Handling Server Response:
 - o Once the server responds, the response.ok is checked to confirm if the product was added successfully.

- o If the response is not OK, an error message is displayed. If the product is successfully created, a confirmation message is shown, and the user is redirected to the product page (/product).
- 6. Logging Server Response:
 - o The console.log(data) line logs the server's response to the console for debugging or informational purposes.
- 7. Error Handling:
 - o If there's an error during the fetch operation, it is caught by the catch() method and logged.

Functions:

1. getSelectedCategory() Function:
 - o This function retrieves the selected category by iterating through radio buttons with the name product-categories. It returns the value of the selected category or null if no category is selected.
2. getSelectedSeason() Function:
 - o Similar to the getSelectedCategory() function, this one retrieves the selected season (Winter or Summer) by checking the product-season radio buttons.
3. Redirect Function (goToProductManagement()):
 - o This function is used to redirect the user to the product management page (/edit) if they choose to cancel the product addition.

/*This is some part of our CSS */

For every page, Use this font:

```
/* General Styles */
body{
  margin: 0;
  font-family: Arial, sans-serif;
}
```

This CSS code sets the basic styling for the entire webpage by targeting the body element. It removes the default browser margin by setting margin: 0, allowing more precise control over the page layout. Additionally, it applies a consistent font across the site by setting the font-family to Arial, with sans-serif as a fallback option, giving the webpage a clean and modern appearance.

For every page, start with this CSS in **header** containing this:

```
.top-banner{
  background-color: #5856d6;
  color: white;
  text-align: center;
  padding: 10px;
}

.top-banner a{
  color: white;
  text-decoration: underline;
}
```

This CSS code styles the top banner of the webpage by giving it a purple background color (#5856d6) and white text, with the content centered horizontally and padding added for better spacing. Additionally, it ensures that any links inside the banner are also white and underlined, making them stand out clearly against the background and easily identifiable as clickable elements.

For every page, start with this CSS in **navigation bar** containing this:

```
/* navigation bar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px 40px;
  background-color: white;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
  position: sticky;
  top: 0;
  z-index: 100;
}
```

This CSS code styles a navigation bar with a flexible layout, ensuring items are spaced evenly and vertically centered. It features a white background, subtle shadow for depth, and sticky positioning to keep it at the top of the page while scrolling. The navbar also has padding for spacing and a high z-index to stay above other content.

```

.logo img {
  height: 50px;
}

.nav-links {
  display: flex;
  gap: 25px;
  list-style: none;
  padding: 0;
  margin: 0;
}

```

This CSS code styles the logo image and the navigation links. The `.logo img` selector sets the height of the logo image to 50px, ensuring it remains a consistent size. The `.nav-links` class uses **flexbox** to arrange the navigation links in a horizontal row, with a 25px gap between each link. It removes the default list styling by setting `list-style: none`, and also eliminates any padding or margin with `padding: 0` and `margin: 0`, creating a clean and organized layout for the links.

```

.nav-links a {
  text-decoration: none;
  color: #333;
  font-weight: 600;
  font-size: 16px;
  transition: color 0.3s ease;
}

.nav-links a:hover {
  color: #5856d6;
}

.nav-icons {
  display: flex;
  gap: 20px;
}

.nav-icons a img {
  height: 20px;
  transition: transform 0.2s ease;
}

.nav-icons a:hover img {
  transform: scale(1.2);
}

```

This CSS code styles the navigation links and icons within the `.nav-links` and `.nav-icons` classes. For the links (`.nav-links a`), it removes the default underline with `text-decoration: none`, sets the text color to a dark gray (#333), and applies a bold font weight of 600 and a font size of 16px. Additionally, it includes a smooth transition effect on the color when the link is hovered over. When hovered, the color changes to a purple shade (#5856d6).

For the navigation icons (.nav-icons), it uses flexbox to arrange them horizontally with a 20px gap between each. The images within the icons are set to a height of 20px, and they have a transition effect on the transform property. When hovered, the images scale up by 1.2 times, creating an interactive visual effect.

For every page, start with this CSS in **footer** containing this:

```
/* footer */
footer{
  background-color: #5a4fcf;
  color: white;
  padding: 30px;
  margin-top: 50px;
}

.footer-container{
  display: flex;
  justify-content: center;
  gap: 50px;
  flex-wrap: wrap;
}

.footer-column{
  max-width: 250px;
}

.footer-column h4{
  font-size: 16px;
  font-weight: bold;
  margin-bottom: 10px;
}

.footer-column ul{
  list-style: none;
  padding: 0;
}

.footer-column ul li{
  margin: 5px 0;
}

.footer-column ul li a{
  color: white;
  text-decoration: none;
  font-size: 14px;
}
```

This CSS code styles the footer section of the webpage, ensuring it has a visually appealing and functional layout. The footer has a purple background color (#5a4fcf), white text, and 30px of padding for spacing. Additionally, there's a 50px margin at the top to create space between the footer and the content above it.

The .footer-container uses **flexbox** to arrange the footer columns horizontally and centers them. The gap: 50px adds space between the columns, and flex-wrap: wrap allows the columns to stack on smaller screens. Each .footer-column has a maximum width of 250px, ensuring the content doesn't stretch too wide.

Inside the footer columns, the heading (h4) is styled to be bold with a font size of 16px and a 10px bottom margin for spacing. The lists within .footer-column ul have no bullet

points, and each list item (li) has a 5px margin for spacing. The links within the list items are white, without underlines, and have a font size of 14px for readability.

```
.footer-column news{
  display: flex;
  margin-top: 10px;
  border: 1px solid white;
  border-radius: 5px;
}

.footer-column news input{
  flex: 1;
  border: none;
  padding: 8px;
  outline: none;
}

.copyright{
  margin-top: 20px;
  font-size: 12px;
  opacity: 0.8;
  text-align: center;
}
```

This CSS code further styles a specific section of the footer, likely intended for a newsletter signup, and the copyright text. The `.footer-column news` class uses **flexbox** to arrange its contents in a row, and it adds a 10px top margin for spacing. It has a white border and rounded corners (`border-radius: 5px`) to give it a clean, polished appearance.

Inside this section, the input field (`.footer-column news input`) is styled to take up all available space with `flex: 1`, ensuring it stretches to fit the container. The input field has no border and no outline on focus (`border: none; outline: none`), and it is padded with 8px for comfort in typing.

Finally, the `.copyright` class styles the copyright notice at the bottom. It adds a 20px top margin, sets the font size to 12px, reduces the opacity to 0.8 for a subtle effect, and centers the text. This ensures the copyright text is less prominent but still legible.

- Team.css

```
.banner {
  background-color: #5856d6;
  color: white;
  padding: 10px;
  text-align: center;
  font-weight: bold;
  font-size: 1.2rem;
}

.team-container {
  text-align: center;
  padding: 40px;
}

.team-header {
  font-size: 2rem;
  font-weight: bold;
  margin-bottom: 20px;
}

.team-image {
  width: 100%;
  max-width: 800px;
  height: auto;
  border-radius: 10px;
  margin-bottom: 40px;
}

.team-info {
  display: flex;
  justify-content: center;
  gap: 30px;
  flex-wrap: wrap;
}

.member-card {
  text-align: center;
  background-color: white;
  padding: 20px;
  border-radius: 10px;
  box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.1);
  transition: transform 0.3s ease;
  width: 250px;
}

.member-card:hover {
  transform: translateY(-10px);
}

.member-card img {
  width: 100%;
  height: 250px;
  object-fit: cover;
  border-radius: 50%;
  margin-bottom: 10px;
}

.member-info {
  font-size: 1rem;
  color: #555;
}

.member-info .email,
.member-info .line {
  display: block;
  margin-top: 5px;
}

.member-info .line {
  font-weight: bold;
}

.member-info a {
  color: #5856d6;
  text-decoration: none;
}

.member-info a:hover {
  text-decoration: underline;
}
```

- The .banner class creates a bold, white text banner with a purple background and centered text.

- The .team-container centers the content and adds padding for spacing.

- The .team-header styles the section title with a larger font size and bold text.

The .team-image class ensures the team image is responsive, with a maximum width of 800px and rounded corners.

- The .team-info class arranges team member cards in a flexible row with space between them and allows wrapping on smaller screens.

- Each .member-card is styled with a white background, padding, and rounded corners, and it includes a subtle shadow and hover effect that lifts the card. The member image is styled to fit within a circular shape, and member details are displayed below the image. Links, such as emails, are styled with the brand's purple color and underlined on hover.

- search.css

```
.search-bar {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: center;
  margin: 30px auto;
  padding: 0 15px;
  max-width: 1200px;
}

.search-bar input {
  padding: 15px 20px;
  font-size: 18px;
  width: 100%;
  max-width: 600px;
  border-radius: 25px;
  outline: none;
  transition: all 0.3s ease;
}

.search-bar button {
  padding: 20px 40px;
  font-size: 22px;
  background-color: #5856d6;
  color: white;
  border: none;
  border-radius: 50px;
  cursor: pointer;
  transition: background-color 0.3s ease;
  margin-top: 20px;
  max-width: 600px;
}

.search-bar button:hover {
  background-color: #af52de;
}

.List {
  display: flex;
  justify-content: center;
  gap: 20px;
  margin: 20px auto;
  padding: 0 15px;
  max-width: 1200px;
}

.category,
.season,
.Sort-Container {
  flex: 1;
  background-color: #ffffff;
  border-radius: 8px;
  padding: 10px;
  transition: all 0.3s ease;
}

.category select,
.season select,
.Sort-Container select {
  width: 100%;
  padding: 12px;
  font-family: Arial, sans-serif;
  font-size: 12px;
  border-radius: 5px;
  border: none;
  background-color: #ffffff;
  transition: background-color 0.3s ease, border-color 0.3s ease;
}

.Category-Container select:focus,
.Season-Container select:focus,
.Sort-Container select:focus {
  border-color: #5856d6;
  box-shadow: 0 0 8px rgba(88, 86, 214, 0.3);
}
```

This CSS styles a search bar and a filter list layout. The `.search-bar` is centered with styled input and button elements, featuring rounded corners, smooth transitions, and hover effects. The `.List` and filter containers like `.category`, `.season`, and `.Sort-Container` use flex layout, padding, and clean white backgrounds. Dropdowns (select) are fully responsive, styled for readability, and highlight with a colored border and shadow when focused for better user interaction.

- Add.css

```
/* Main Form Section */
.Add-product {
  padding: 40px 20px;
  max-width: 800px;
  margin: 0 auto;
  background-color: #white;
  border-radius: 8px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
}

.Add-product-header h1 {
  font-size: 28px;
  color: #333;
  text-align: center;
  margin-bottom: 30px;
}

.Add-product-container {
  display: flex;
  flex-direction: column;
}

.product-details {
  display: flex;
  flex-direction: column;
}

.Label {
  margin-bottom: 15px;
}

.Label label {
  display: block;
  font-weight: bold;
  margin-bottom: 5px;
  color: #333;
}

.Label input, .Label textarea {
  width: 100%;
  padding: 10px;
  border-radius: 4px;
  border: 1px solid #ddd;
  font-size: 16px;
}

.Label textarea {
  resize: vertical;
}

.option {
  margin-bottom: 20px;
}

.option label {
  font-size: 16px;
  margin-right: 10px;
}

.option input {
  margin-right: 10px;
}

.bottom-container {
  display: flex;
  justify-content: space-between;
  gap: 15px;
}

.bottom-container button {
  padding: 12px 20px;
  font-size: 16px;
  cursor: pointer;
  border-radius: 4px;
  border: none;
  transition: background-color 0.3s;
}
```

```
.bottom-container button[type="reset"] {
  background-color: #f44336;
  color: white;
}

.bottom-container button[type="submit"] {
  background-color: #4CAF50;
  color: white;
}

.bottom-container button[type="reset"]:hover {
  background-color: #d32f2f;
}

.bottom-container button[type="submit"]:hover {
  background-color: #388e3c;
}
```

This CSS code styles a product addition form with a clean, modern layout. It centers the form with a white background, rounded corners, and a subtle shadow. Input fields are styled with padding, borders, and clear labels. Buttons are designed with distinct colors for "reset" and "submit," with hover effects for interactivity. The form is organized vertically, ensuring good spacing and readability.

- Detailpage.css

```
.banner {
  background-color: #5856d6;
  color: white;
  padding: 10px;
  text-align: center;
  font-weight: bold;
}

.main-content {
  margin-left: 220px;
  padding: 20px;
}

.product-container {
  display: flex;
  max-width: 1100px;
  margin: 50px auto;
  gap: 50px;
}

.product-gallery {
  margin: 30px;
  justify-content: space-between;
  /*width: 95%;*/
  overflow: hidden;
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
  margin-top: 20px;
}

.product-gallery img {
  width: 100%;
  border-radius: 5px;
}

.details-pants-right {
  display: block;
  gap: 10px;
  margin-top: 10px;
  float: right;
  width: 65%;
}

.details-pants-right img {
  width: 105%;
  height: auto;
  border-radius: 5px;
}

.details-pants-left {
  display: block;
  gap: 10px;
  margin-top: 10px;
  float: left;
  width: 25%;
}

.details-pants-left img {
  display: block;
  width: 90%;
  border-radius: 5px;
  margin: auto;
}

.sizes,
.counter {
  margin-bottom: 10px;
}

button {
  padding: 8px 12px;
  margin: 5px;
  cursor: pointer;
  border: 1px solid #000;
  background-color: #fff;
  border-radius: 5px;
  font-size: 16px;
}

button:hover {
  background-color: #6b59d4;
}
```

This CSS styles a product detail and gallery page with a modern, user-friendly layout. The `.banner` provides a bold, centered heading with white text on a purple background. The `.main-content` is spaced to the right, likely to accommodate a sidebar. The `.product-container` uses a flex layout to arrange product images and details side by side, while the `.product-gallery` wraps and spaces out multiple product images.

```

.counter button {
  font-size: 20px;
  width: 40px;
  height: 40px;
}

.button-container {
  display: flex;
  align-items: center;
  gap: 10px;
}

.buy-now {
  background-color: #635DFF;
  color: white;
  border: none;
  padding: 12px 20px;
  font-size: 16px;
  border-radius: 8px;
}

.buy-now:hover {
  background-color: #4a46cc;
}

.favorite-btn {
  width: 50px;
  height: 50px;
  border: 2px solid #ccc;
  border-radius: 10px;
  background-color: white;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
  transition: all 0.3s ease;
}

.favorite-btn svg {
  width: 24px;
  height: 24px;
  fill: none;
  stroke: black;
  stroke-width: 2;
  transition: fill 0.3s ease, stroke 0.3s ease;
}

.favorite-btn:hover {
  background-color: #c486a5;
}

.container {
  display: flex;
  flex-direction: column;
  align-items: flex-end;
}

.related-title {
  padding-left: 30px;
}

.section {
  display: flex;
  align-items: center;
  justify-content: center;
  width: 300px;
  height: 100px;
  display: flex;
  border: 1px solid #ccc;
}

.icon {
  margin-right: 15px;
}

a {
  text-decoration: none;
  color: #0000EE;
}

.section:hover {
  background-color: #f0f0f0;
}

```

```

.product-card {
  width: 180px;
  text-align: center;
}

.product-card img {
  width: 100%;
  height: auto;
}

.product-card p {
  font-size: 8px;
  font-weight: bold;
  color: #333;
  margin-top: 10px;
}

```

The left and right detail sections (`.details-pants-left` and `.details-pants-right`) handle product images responsively, floating to each side with appropriate widths. Buttons are styled consistently with padding, borders, hover effects, and specific styles for "buy now" and counter buttons. The `.favorite-btn` offers an interactive heart icon with hover transitions. Layout elements like `.container`, `.section`, and `.related-title` organize content neatly, with centered alignment and hover effects. Lastly, `.product-card` styles individual items in the gallery, giving them compact, readable designs.

- Edit.css

```
/* Main Content */
main {
  padding: 20px;
  max-width: 1200px;
  margin: auto;
}

h1, h2 {
  font-family: serif;
  font-weight: 200;
  text-align: left;
  color: #333;
}

.products, .more-items {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 20px;
  justify-items: center;
  margin-top: 20px;
}

.product {
  background: white;
  padding: 15px;
  text-align: center;
  border-radius: 10px;
  box-shadow: 0 2px 5px #000000;
  transition: transform 0.3s ease-in-out;
}

button img {
  width: 25px;
  height: 25px;
  vertical-align: middle;
}

.product img {
  width: 100%;
  max-width: 180px;
}

.product p {
  font-family: sans-serif;
  font-weight: 300;
  margin: 10px 0;
  color: #333;
}

button {
  display: inline;
  background-color: transparent;
  border: none;
}

button:hover {
  background-color: #cdcdcd;
}

.cart {
  border: none;
  padding: 12px;
  color: white;
  background-color: black;
  justify-content: center;
  text-align: center;
  cursor: pointer;
  width: 100%;
  font-size: 10px;
  cursor: pointer;
  transition: 0.3s;
}

.add {
  width: 100%;
  padding: 20px;
  justify-content: center;
}
```

This CSS code styles a webpage featuring a product grid, a shopping cart, and a popup interface. The main content is centered with a maximum width and padding. Product items are displayed in a responsive grid with a light design, rounded corners, and subtle shadows. Buttons, including cart and action buttons, have interactive styles with hover effects. A popup is hidden by default and appears centered with a semi-transparent background overlay when triggered. The popup has a clear design with confirm and cancel buttons styled for visibility and interaction.


```

.popup {
  display: none; /* Hidden by default */
  width: 500px;
  height: 100px;
  font-weight: 400;
  font-size: xx-large;
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: #230, 229, 229);
  padding: 50px;
  border: none;
  border-radius: 30px;
  box-shadow: 0px 0px 10px rgba(0, 0, 0, 0.3);
  z-index: 1000;
  text-align: center;
}

/* Background overlay */
.overlay {
  display: none; /* Hidden by default */
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.5);
  z-index: 999;
}

.cancel {
  display: inline;
  margin-top: 10px;
  padding: 5px 10px;
  background: white;
  color: red;
  border: 5px #7d7d7d;
  border-radius: 5px;
  cursor: pointer;
  font-weight: bold;
}

.confirm {
  display: inline;
  margin-top: 10px;
  padding: 5px 10px;
  background: white;
  color: rgb(20, 179, 15);
  border: 5px #7d7d7d;
  border-radius: 5px;
  cursor: pointer;
  font-weight: bold;
}

```

The CSS code provided is used to design a **popup modal** with a **background overlay** and interactive **cancel** and **confirm** buttons. The popup is initially hidden and will appear centered on the screen with a light gray background, rounded corners, and a subtle shadow effect for visual depth. It is positioned using fixed placement to stay in place even when scrolling, and it includes padding and centered text. The background overlay also covers the entire viewport with a semi-transparent black color to dim the background when the popup is active. The **cancel** and **confirm** buttons are displayed inline, with the cancel button styled with a white background and red text, while the confirm button uses a white background with green text. Both buttons have a bold font weight, rounded borders, and a pointer cursor to indicate they are interactive. This CSS ensures a user-friendly and visually appealing modal interaction for the web application.

- Homepage.css

```
.main-content {
  margin-left: 0;
  padding: 0 100px;
}

.banner img{
  width: 100%;
}

.best-selling h5{
  margin-bottom: 10px;
  color: #5856d6;
}

.new-arrivals h5{
  margin-bottom: 10px;
  color: #5856d6;
}

.section-label {
  font-size: 14px;
  font-weight: 600;
  color: #db4444;
  display: flex;
  align-items: center;
  gap: 10px;
  margin-bottom: 10px;
}

.section-label::before {
  content: '';
  display: inline-block;
  width: 10px;
  height: 20px;
  background-color: #5856d6;
  border-radius: 2px;
}

.view-products{
  display: block;
  margin: 20px auto;
  padding: 10px 20px;
  background-color: #af52de;
  color: white;
  border: none;
  cursor: pointer;
}

.features{
  display: flex;
  justify-content: space-around;
  margin-top: 20px;
  text-align: center;
}

.icon img{
  width: 40px;
  height: 40px;
}

.features-text h3{
  font-size: 18px;
  font-weight: bold;
  margin: 10px 0;
}

.features-text p{
  font-size: 14px;
  color: gray;
}
```

```
.best-selling-grid,
.new-arrivals-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 20px;
  justify-content: center;
  margin-top: 20px;
}

@media (min-width: 1200px) {
  .best-selling-grid,
  .new-arrivals-grid {
    grid-template-columns: repeat(5, 1fr);
  }
}

.product-card1,
.product-card2 {
  background-color: white;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
  border-radius: 8px;
  overflow: hidden;
  text-align: center;
  transition: transform 0.2s;
  padding: 10px;
}

.product-card1:hover,
.product-card2:hover {
  transform: translateY(-5px);
}

.see-more-button {
  margin-top: 10px;
  background-color: #AF52DE;
  color: white;
  padding: 5px 10px;
  border-radius: 5px;
  font-size: 14px;
  cursor: pointer;
}

.product-name {
  font-size: 16px;
  margin: 10px 0 5px 0;
}

.product-price {
  font-weight: bold;
  color: #ff4081;
  margin-bottom: 10px;
  font-size: 14px;
}

.product-card button {
  border: none;
  outline: 0;
  padding: 12px;
  color: white;
  background-color: #5856d6;
  text-align: center;
  cursor: pointer;
  width: 100%;
  font-size: 10px;
}
```

This CSS designs a product showcase and footer for a website. The `.main-content` sets padding, and the `.banner` ensures full-width banner images. Sections like `.best-selling` and `.new-arrivals` have styled headings, and `.section-label` adds a colored label marker. Product grids use responsive layouts with hover effects (`.product-card1`, `.product-card2`) and a "see more" button. Features are displayed with icons and text. The footer uses a flexible grid layout (`.footer-container`) with styled links and a newsletter input. Buttons like `.view-products-button` are styled with background colors, hover effects, and centered alignment for better user interaction.

- Product.css

```
.search-login {
  display: flex;
  align-items: center;
  gap: 15px;
}

.search-login input {
  padding: 6px 10px;
  border-radius: 6px;
  border: 1px solid #ccc;
}

.product-list-container {
  max-width: 1200px;
  margin: 40px auto;
  padding: 0 20px;
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(250px, 1fr));
  gap: 50px 30px;
}

.column {
  background-color: #fff;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.08);
  transition: transform 0.2s ease;
  cursor: pointer;
}

.column:hover {
  transform: translateY(-5px);
}

.image-container img {
  width: 100%;
  height: 250px;
  object-fit: cover;
}

.product-details {
  padding: 15px;
}

.product-title {
  font-size: 18px;
  margin-bottom: 5px;
}

.product-price {
  font-size: 16px;
  font-weight: bold;
  color: #ff4081;
  margin-bottom: 10px;
}

.product-description {
  font-size: 14px;
  color: #666;

  height: 60px;
  overflow: hidden;
  text-overflow: ellipsis;
  display: -webkit-line-clamp 3;
}
```

This CSS styles a product listing page with a flexbox search bar and a responsive grid layout. Product cards (.column) have shadows, rounded corners, and a hover lift effect. Images are uniform in size, and product details like title, price, and a truncated description are neatly formatted for a clean.

- Search.css

```
.product-list-container {
  padding: 20px;
}

.Row {
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  margin-bottom: 20px;
}

.banner {
  height: 150px;
  margin-bottom: 20px;
}

.column {
  flex: 1 1 calc(25% - 20px);
  background: white;
  border: 1px solid #eee;
  border-radius: 10px;
  overflow: hidden;
  cursor: pointer;
  transition: transform 0.2s;
}

@media (max-width: 992px) {
  .column {
    flex: 1 1 calc(50% - 20px);
  }
}

@media (max-width: 576px) {
  .column {
    flex: 1 1 100%;
  }
}

.main-content {
  margin-left: 0;
  padding: 0 100px;
}

.column:hover {
  transform: translateY(-5px);
}

.image-container img {
  width: 100%;
  height: auto;
}

.product-details {
  padding: 10px;
}

.product-title {
  margin: 0;
  font-size: 18px;
  font-weight: bold;
}

.product-price {
  margin: 5px 0;
  color: #e91e63;
  font-weight: bold;
}

.List {
  display: flex;
  gap: 20px;
  padding: 20px;
  flex-wrap: wrap;
}

.category, .season {
  flex: 1;
}

.Sort-Container {
  padding: 5px;
  border-radius: 5px;
  border: 1px solid #ccc;
}
```

This CSS styles a responsive product listing page. It uses flexbox to arrange product cards (.Row, .column) and adjusts layout for different screen sizes. Product cards have a hover effect and clean design with padding, borders, and responsive images. The .banner sets the banner height, while .product-title and .product-price format text. Filter sections (.List, .category, .season, .Sort-Container) are spaced and aligned for a user-friendly layout.

- Sign-in.css

```
.banner {
  background-color: #5856d6;
  color: white;
  padding: 10px;
  text-align: center;
  font-weight: bold;
}

.login-container {
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 60px 40px;
  gap: 60px;
  flex-wrap: wrap;
  background-color: #f8f8ff;
}

.logo-container {
  max-width: 500px;
}

.logo-container img {
  width: 100%;
  border-radius: 16px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.signin-signup {
  background-color: white;
  padding: 40px;
  border-radius: 16px;
  box-shadow: 0 4px 16px rgba(0, 0, 0, 0.1);
  max-width: 400px;
  width: 100%;
}

.sign-in-form .title {
  font-size: 28px;
  font-weight: bold;
  margin-bottom: 30px;
  color: #000000;
  text-align: center;
}

.form-row {
  margin-bottom: 20px;
}

.form-control {
  width: 100%;
  padding: 12px;
  font-size: 16px;
  border: 1px solid #ccc;
  border-radius: 8px;
  box-sizing: border-box;
  transition: border-color 0.3s ease;
}

.form-control:focus {
  border-color: #6c63ff;
  outline: none;
}

.checkbox {
  display: flex;
  align-items: center;
  gap: 8px;
  font-size: 14px;
  margin-bottom: 20px;
}

.sign-in.button {
  width: 100%;
  padding: 12px;
  font-size: 16px;
  font-weight: bold;
  background-color: #6c63ff;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

.sign-in.button:hover {
  background-color: #5a52e8;
}

.message {
  margin-top: 20px;
  color: red;
  text-align: center;
  font-size: 14px;
}
```

This CSS styles a clean and modern login page. The banner creates a colored header with centered bold text. The .login-container uses flexbox to center the login form and logo, with responsive wrapping. The logo-container holds a styled image with rounded corners and shadow. The signin-signup box styles the login form with padding, rounded edges, and a soft shadow. Inside, form elements (form-control) are fully responsive with focus effects. The sign-in.button is a bold, purple button with hover color change. The message class styles error or status messages in red, centered below the form.

- view.css

```
main {
  font-family: sans-serif;
  font-weight: 300;
  text-align: center;
  padding: 40px 20px;
}
button img {
  display: flex;
  justify-content: end;
  max-width: 30px;
  vertical-align: middle;
}
button {
  display: inline;
  background-color: transparent;
  border: none;
}
button:hover {
  background-color: #cdcdcd;
}
.product-list {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 20px;
}
.product-item {
  display: flex;
  align-items: center;
  background: white;
  padding: 15px;
  border-radius: 8px;
  width: 60%;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}
.product-item img {
  width: 250px;
  height: auto;
  margin-right: 20px;
}
.product-info h2, .product-info p {
  display: flex;
  justify-content: flex-start;
  font-family: sans-serif;
  font-weight: 300;
  font-size: 16px;
  text-align: left;
}
.add-button {
  border-color: transparent;
  background-color: transparent;
  margin-top: 20px;
}
.save-button {
  background-color: #d8d8d8;
  color: black;
  border: none;
  padding: 10px 20px;
  border-radius: 5px;
  cursor: pointer;
  margin-top: 20px;
  font-size: 18px;
}
.save-button {
  display: block;
  margin: 20px auto;
}
```

This CSS styles a simple product list page. The main section centers content with padding and light font styling. Buttons are styled transparently with a hover background effect. The product-list arranges products vertically with spacing, and each .product-item displays as a card with an image and text side-by-side. Product details are aligned to the left. The .add-button is minimal, while the .save-button is styled for visibility with a gray background, padding, and centered positioning.

File JS

cookie.js

```
function setCookie(cname, cvalue, minutes) {  
  
    let now = new Date();  
    let dUTC = new Date(now.getTime() + (minutes * 60 * 1000));  
  
    let expires = "expires=" + dUTC.toUTCString();  
    document.cookie = cname + "=" + encodeURIComponent(cvalue) + ";" + expires + ";path=/";  
  
}
```

The setCookie function is used to create a cookie in the user's browser with a specified name, value, and expiration time in minutes. It calculates the expiration time by adding the given minutes to the current time, formats it in UTC, and saves the cookie using document.cookie. The cookie is accessible across the entire website (path=/), and the value is safely encoded using encodeURIComponent.

```
function getCookie(cname) {  
    let name = cname + "=";  
    let decodedCookie = decodeURIComponent(document.cookie);  
  
    let ca = decodedCookie.split(';');  
    for (let i = 0; i < ca.length; i++) {  
        let c = ca[i];  
        while (c.charAt(0) == ' ') {  
            c = c.substring(1);  
        }  
        if (c.indexOf(name) == 0) {  
            return c.substring(name.length, c.length);  
        }  
    }  
    return "";  
}
```

The getCookie function is used to retrieve the value of a specific cookie from the user's browser. It accepts the cookie name (cname) as input. The function first decodes the document.cookie string to ensure special characters are correctly interpreted. It then splits the cookies into an array and iterates through them. For each cookie, it removes any leading spaces and checks if it starts with the target cookie name. If found, it returns the corresponding cookie value. If the cookie does not exist, it returns an empty string.

```
function deleteCookie(cname) {  
    document.cookie = cname + "=" + expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;";  
  
}
```

The deleteCookie function is used to remove a specific cookie from the user's browser. It does this by setting the cookie's value to an empty string ("") and its expiration date to a time in the past (01 Jan 1970). By setting the expires attribute to a date that has already passed, the browser automatically deletes the cookie. The path=/ ensures that the deletion applies across the entire website.

Details of your web services and code

backend.js

```
const express = require("express");
const path = require("path");
const app = express();
const router = express.Router();
app.use("/", router);
app.use(express.static(path.join(__dirname, 'Front-end')));
router.use(express.json());
router.use(express.urlencoded({ extended: true }));

const dotenv = require("dotenv");
dotenv.config();

const cors = require("cors");
/* This is for specific origin and method */
let corsOptions = {
  origin: 'http://localhost:3300',
  methods: 'GET,POST,PUT,DELETE'
}
router.use(cors(corsOptions));

const session = require("express-session");
app.use(session({
  secret: 'secret',
  resave: true,
  saveUninitialized: true
}));

const cookieParser = require('cookie-parser');
router.use(cookieParser('MY Cookie'));
```

This backend.js code sets up a basic Express server:

- **Express and Router Setup:** It imports express and path, creates an app and router, and configures routing at the root (/).
- **Static Files:** It serves static files from the Front-end folder.
- **Body Parsing:** It enables parsing of JSON and URL-encoded request bodies.
- **Environment Variables:** dotenv is used to load environment variables from a .env file.
- **CORS Configuration:** It restricts API access to only requests from <http://localhost:3300> using specific HTTP methods (GET, POST, PUT, DELETE).
- **Session Management:** It uses express-session to manage user sessions, storing session data securely.
- **Cookie Parsing:** It uses cookie-parser to handle cookies with the secret key 'MY Cookie'.

```

/* Connection to MySQL */
const mysql = require("mysql2");
var connection = mysql.createConnection({
  host: process.env.MYSQL_HOST,
  user: process.env.MYSQL_USERNAME,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE
});

/* Connection to DB */
connection.connect(function (err) {
  if (err) {
    throw err;
  }
  console.log(`Connected DB : ${process.env.MYSQL_DATABASE}`)
})

```

This code establishes a connection to a MySQL database using the mysql2 library.

- It creates a connection object with database credentials (host, username, password, database name) loaded from environment variables (process.env).
- Then, it attempts to connect to the database.
- If the connection fails, it throws an error. If successful, it logs a message showing the connected database name.

```

/* FOR HOMEPAGE PAGE*/
router.get("/new-products", (req, res) => {
  connection.query('SELECT * FROM Product WHERE prod_status = "NEW" LIMIT 5', (error, results) => {
    if (error) {
      console.error('Error fetching products:', error);
      return res.status(500).json({ error: 'Internal Server Error' });
    }
    res.json(results);
  });
});

router.get("/best-selling", (req, res) => {
  connection.query('SELECT * FROM Product WHERE prod_category = "Women" AND prod_seasons = "Summer" LIMIT 5', (error, results) => {
    if (error) {
      console.error('Error fetching products:', error);
      return res.status(500).json({ error: 'Internal Server Error' });
    }
    res.json(results);
  });
});

```

This code defines two API routes using Express for the homepage:

- /new-products:
It queries the database for up to 5 products where the product status is "NEW" and sends them as a JSON response.
- /best-selling:
It queries the database for up to 5 products where the category is "Women" and the season is "Summer", then returns them as JSON.

```
/* FOR PRODUCT PAGE */
router.get("/products", (req, res) => {
  alert('product');
  connection.query('SELECT * FROM Product', (error, results) => {
    if (error) {
      console.error('Error fetching products:', error);
      return res.status(500).json({ error: 'Internal Server Error' });
    }
    res.json(results);
  });
});
```

This code creates an API route /products for the product page.

When accessed, it:

- Fetches all products from the Product table in the database.
- Returns the product data as a JSON response.
- If an error occurs during the database query, it sends a 500 Internal Server Error response.

```

/* FOR SEARCH PAGE */
// Endpoint to fetch product data from the database
router.get('/select', (req, res) => {
  let sql = 'SELECT * FROM Product WHERE 1 = 1'; // Initial query

  // Check if productName query parameter is provided
  if (req.query.productName) {
    const productName = req.query.productName;
    sql += ` AND prod_name LIKE '%${productName}%'`;
  }

  // Check if category query parameter is provided
  if (req.query.category && req.query.category !== 'All Products') {
    const category = req.query.category;
    sql += ` AND prod_category = '${category}'`;
  }

  // Check if season query parameter is provided
  if (req.query.season && req.query.season !== 'All Products') {
    const season = req.query.season;
    sql += ` AND prod_seasons = '${season}'`;
  }

  // Check if sortBy query parameter is provided
  if (req.query.sortBy) {
    const sortBy = req.query.sortBy;
    switch (sortBy) {
      case 'nameAZ':
        sql += ' ORDER BY prod_name ASC';
        break;
      case 'nameZA':
        sql += ' ORDER BY prod_name DESC';
        break;
      case 'priceLowToHigh':
        sql += ' ORDER BY prod_price ASC';
        break;
      case 'priceHighToLow':
        sql += ' ORDER BY prod_price DESC';
        break;
      default:
        return res.status(400).json({ error: true, message: 'Invalid sortBy option' });
    }
  }
}

```

```

// Execute the query
connection.query(sql, (error, results) => {
  if (error) {
    console.error('Error fetching product data:', error);
    return res.status(500).json({ error: true, message: 'Internal server error' });
  }
  return res.json(results);
});
});

```

This code defines a route handler in an Express.js application for handling GET requests to the /select endpoint. Here's a breakdown:

1. Initial SQL Query: The query starts by selecting all records from the Product table (SELECT * FROM Product WHERE 1 = 1).

2. Query Parameters:

- o productName: If a productName is provided in the request, it adds a filter to the query using a LIKE operator to search for products whose names match the given value.
 - o category: If a category is provided and is not 'All Products', it filters the products by the specified category.
 - o season: If a season is provided and is not 'All Products', it filters the products by the specified season.
 - o minPrice & maxPrice: If both minPrice and maxPrice are provided, it adds a price range filter using the BETWEEN operator.
3. Sorting: If a sortBy query parameter is provided, the products are ordered based on the given criteria (nameAZ, nameZA, priceLowToHigh, or priceHighToLow). If the sortBy option is invalid, it returns a 400 error.
4. Query Execution: The final SQL query is executed using the connection.query method. If there's an error, a 500 error response is returned. Otherwise, the results are sent as a JSON response.

```
/* FOR LOGIN PAGE*/
router.post("/form-post", (req, res) => {

  const username = req.body.username;
  const password = req.body.password;

  let sql = `SELECT * FROM Login WHERE (login_email = ?) AND (login_password = ?)`;
  console.log('sql: '+sql);
  connection.query(sql, [username, password], function (error, results) {
    if (error) {
      res.status(500).send("Internal server error. Please try again."); // Internal server error
    } else {
      console.log(results.length);
      if (results.length > 0) {
        // Set cookie upon successful login
        return res.status(200).send({ error: false, message: "true" });

        console.log("Sign In");
      } else {
        // Render HTML with pop-up message for invalid credentials
        return res.status(401).send({ error: true, message: "Invalid username or password. Please try again." });
        console.log("Sign In Failed");
      }
    }
  })
})
})
```

This code handles the login functionality for a web application. It receives a POST request with the user's username and password. The credentials are checked against the Login table in the database. If a matching record is found, a success response is sent; otherwise, an error message indicating invalid credentials is returned. If there is a query error, a server error

message is sent. The code also logs the SQL query and query result length for debugging purposes.

```
/* FOR PRODUCT DETAIL PAGE */
router.get("/product_detail", (req, res) => {

  const productId = req.query.prod_ID;
  // Validate product ID parameter
  if (!productId) {
    return res.status(400).json({ error: 'product ID parameter is required' });
  }

  // Fetch product details from the database based on SKU
  connection.query('SELECT * FROM Product WHERE prod_ID = ?', [productId], (error, results) => {
    if (error) {
      console.error('Error fetching product details:', error);
      return res.status(500).json({ error: 'Internal Server Error' });
    }

    // Check if product details were found
    if (results.length === 0) {
      return res.status(404).json({ error: 'Product details not found' });
    }

    // Send the product details to the client
    const productDetails = results[0]; // Assuming only one product matches the SKU
    console.log(productDetails);
    res.json(productDetails);
  })
})
```

This code defines a route handler for the `/product_detail` endpoint in an Express.js application, which is used to fetch and return the details of a specific product. Here's an explanation:

1. Extracting the Product ID:
 - o The `productId` is retrieved from the query parameters (`req.query.prod_ID`).
2. Validating the Product ID:
 - o If the `productId` is not provided, a 400 status code is returned, along with an error message indicating that the product ID is required.
3. Fetching Product Details:
 - o If the `productId` is provided, a query is executed to fetch the product details from the Product table in the database using the provided `productId`.
4. Handling Query Results:
 - o If there's an error executing the query, a 500 status code is returned, indicating a server error.
 - o If no product is found with the given `productId` (i.e., the query returns no results), a 404 status code is returned, indicating that the product details were not found.
 - o If the product is found, the details are extracted from the query results (assuming only one product matches the ID) and returned as a JSON response.


```

// Endpoint to fetch product data from the database based on season
router.get('/select_season', (req, res) => {
  const category = req.query.season; // Get the category from the query parameter
  let sql;
  if (category) {
    // If a category is provided, filter products based on that season
    sql = `SELECT * FROM Product WHERE prod_seasons = ?`;
  } else {
    // If no category is provided, fetch all products
    sql = 'SELECT * FROM Product';
  }

  // Execute the query with or without category filtering
  connection.query(sql, [category], (error, results) => {
    if (error) {
      console.error('Error selecting products:', error);
      return res.status(500).json({ error: true, message: 'Internal server error' });
    }
    return res.json(results);
  })
})

```

This code fetches products based on the provided season. If a season is given, it filters products by that season; otherwise, it returns all products. The results are returned as a JSON response. If an error occurs, a 500 error is returned.

```

// Endpoint to fetch product data from the database
router.get('/select_sort', (req, res) => {
  let sql = 'SELECT * FROM Product';

  // Check if sortBy query parameter is provided
  if (req.query.sortBy) {
    const sortBy = req.query.sortBy;
    switch (sortBy) {
      case 'nameAZ':
        sql += ' ORDER BY prod_name ASC';
        break;
      case 'nameZA':
        sql += ' ORDER BY prod_name DESC';
        break;
      case 'priceLowToHigh':
        sql += ' ORDER BY prod_price ASC';
        break;
      case 'priceHighToLow':
        sql += ' ORDER BY prod_price DESC';
        break;
      default:
        // Handle invalid sortBy option
        return res.status(400).json({ error: true, message: 'Invalid sortBy option' });
    }
  }

  // Execute the query
  connection.query(sql, (error, results) => {
    if (error) {
      console.error('Error fetching product data:', error);
      return res.status(500).json({ error: true, message: 'Internal server error' });
    }
    return res.json(results);
  });
});

```

This code fetches product data from the database, applying optional sorting based on the sortBy query parameter. The sorting can be by name (ascending or descending) or by price (low to high or high to low). The results are returned as a JSON response.


```

router.get('/options', (req, res) => {
  connection.query('SELECT DISTINCT prod_category FROM Product', (error, results) => {
    if (error) {
      console.error('Error fetching categories:', error);
      res.status(500).json({ error: true, message: 'Internal server error' });
    } else {
      const categories = results.map(row => row.category);
      res.json({ categories });
    }
  })
})

```

This code fetches distinct product categories from the database and returns them as a JSON array. If an error occurs during the query, a 500 error is returned.

```

router.get('/categories', (req, res) => {
  // Query to fetch category data from the database
  const sql = 'SELECT DISTINCT prod_category FROM Product';

  // Execute the query
  connection.query(sql, (err, results) => {
    if (err) {
      console.error('Error fetching categories:', err);
      res.status(500).json({ error: 'Error fetching categories' });
      return;
    }

    // Process the results and structure them in the desired format
    const categories = results.map(row => row.prod_category);

    // Send the structured data as the response
    res.json({ categories });
  })
})

```

This code defines a route handler for the /categories endpoint, which is used to fetch distinct product categories from the database. The query `SELECT DISTINCT prod_category FROM Product` is executed to retrieve all unique product categories from the Product table. If there is an error during the execution of the query, a 500 error response is returned, along with an error message. If the query is successful, the results are processed by extracting the `prod_category` field from each row, and the distinct categories are returned as a JSON response in the format `{ categories: [...] }`. This response provides a structured list of product categories to the client.

```

router.get('/season', (req, res) => {
  // Query to fetch category data from the database
  const sql = 'SELECT DISTINCT prod_seasons FROM Product';

  // Execute the query
  connection.query(sql, (err, results) => {
    if (err) {
      console.error('Error fetching categories:', err);
      res.status(500).json({ error: 'Error fetching categories' });
      return;
    }

    // Process the results and structure them in the desired format
    const categories = results.map(row => row.prod_category);

    // Send the structured data as the response
    res.json({ categories });
  })
})

```

This code defines a route handler for the /season endpoint, which fetches distinct product seasons from the database. The SQL query `SELECT DISTINCT prod_seasons`

FROM Product retrieves all unique product seasons from the Product table. If there is an error executing the query, a 500 error response is returned with an error message. When the query is successful, the results are processed by extracting the prod_seasons field from each row. However, there is an issue in the code where the categories variable is incorrectly populated with prod_category instead of prod_seasons. The structured data is then returned as a JSON response in the format { categories: [...] }, which provides a list of distinct product seasons to the client.

Insert Product

```
/* Insert product */
router.post("/add-product", function (req, res) {
  const Product = req.body.Product;
  console.log(Product);

  if (!Product.prod_ID || !Product.prod_name || !Product.prod_status) {
    return res.status(400).send({
      error: true, message: 'Please provide product information'
    });
  }

  const sql = `INSERT INTO Product (prod_ID, prod_name, prod_status, prod_pic1, prod_pic2, prod_pic3, prod_pic4, prod_category, prod_seasons, prod_description, prod_price, prod_quantity)
  VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;

  connection.query(sql, [
    Product.prod_ID,
    Product.prod_name,
    Product.prod_status,
    Product.prod_pic1,
    Product.prod_pic2,
    Product.prod_pic3,
    Product.prod_pic4,
    Product.prod_category,
    Product.prod_seasons,
    Product.prod_description,
    Product.prod_price,
    Product.prod_quantity
  ], function (error, results) {
    if (error) {
      console.error("Error inserting product:", error);
      return res.status(500).send({
        error: true,
        message: error//'Error inserting product into the database'
      });
    }

    // Send success message
    return res.send({
      error: false,
      message: 'New product has been created successfully.',
      data: results
    });
  });
});
```

This code defines a route handler for the /add-product endpoint that handles a POST request to add a new product to the database. It first retrieves the Product data from the request body. It checks if essential product details such as prod_ID, prod_name, and prod_status are provided. If any of these fields are missing, it responds with a 400 error, indicating that the product information is incomplete.

If the required data is present, the code constructs an SQL query to insert the new product into the Product table. The SQL query includes placeholders for values such as product ID, name, status, images, category, season, description, price, and quantity. These values are provided as parameters in the query to prevent SQL injection.

The query is executed using connection.query. If the query results in an error, a 500 server error is returned with an error message. If the insertion is successful, a success message is sent back to the client along with the inserted product data. This ensures that the new product is added to the database and the client receives confirmation of the action.

Delete Product

```
/* DELETE product */
router.delete('/delete_product', (req, res) => {

  const sku = req.query.sku; // Get the product SKU from the query parameter

  if (!sku) {
    return res.status(400).json({ error: true, message: 'Product SKU is required' });
  }

  // Query to delete the product from the database
  const sql = 'DELETE FROM Product WHERE prod_ID = ?';

  // Execute the delete query
  connection.query(sql, [sku], (error, results) => {
    if (error) {
      console.error('Error deleting product:', error);
      return res.status(500).json({ error: true, message: 'Internal server error' });
    }
    return res.status(200).json({ success: true, message: 'Product deleted successfully' });
  })
});
```

This code defines an API endpoint for deleting a product in a Node.js application using Express. The `router.delete('/delete_product', (req, res) => {...})` listens for DELETE requests at the `/delete_product` URL. It first retrieves the `sku` (Stock Keeping Unit) from the query parameters of the request (`req.query.sku`). If no SKU is provided, the server immediately responds with a 400 Bad Request error and a message indicating that the SKU is required. If a SKU is present, it prepares a SQL DELETE statement (`DELETE FROM Product WHERE prod_ID = ?`) to remove the product matching the given `prod_ID`. It then executes the SQL query using the database connection (`connection.query`). If the query fails, it responds with a 500 Internal Server Error; otherwise, it returns a 200 OK response, indicating the product was successfully deleted.

Update Product

```
/* UPDATE product */
router.post('/update-product', (req, res) => {

  const prod_ID = req.query.prod_ID;
  console.log(prod_ID);

  if (!prod_ID) {
    return res.status(400).send({
      error: true, message: 'Please provide product information'
    });
  }

  const updatedProduct = req.body.Product; // Get the updated product data from the request body
  console.log(updatedProduct);

  if (!prod_ID || Object.keys(updatedProduct).length === 0) {
    return res.status(400).send({ error: true, message: 'Please provide a Product and updated product data' });
  }

  connection.query('UPDATE Product SET ? WHERE prod_ID = ?', [updatedProduct, prod_ID], (error, results) => {
    if (error) {
      console.error('Error updating product:', error);
      return res.status(500).send({ error: true, message: 'Error updating product' });
    }
    return res.send({ error: false, data: results.affectedRows, message: 'Product updated successfully' });
  });
});
```

This code defines a POST API endpoint `/update-product` using Express to update product information in the database. It first retrieves the `prod_ID` from the query parameters (`req.query.prod_ID`) and logs it for debugging. If `prod_ID` is not provided, the server responds with a 400 Bad Request and an error message. It then retrieves the updated product data from the request body (`req.body.Product`) and logs it. If either the `prod_ID` is missing or the updated product data is empty, it again sends a 400 Bad Request response. If all required

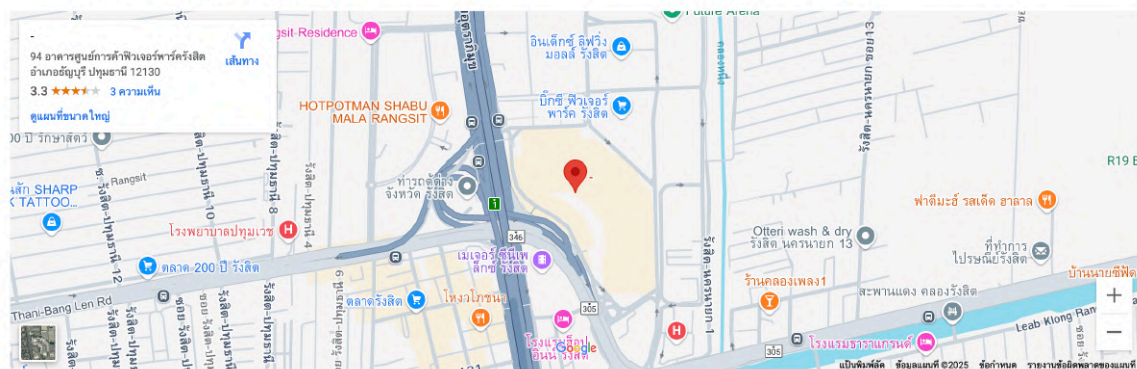
data is available, it constructs an SQL UPDATE query (UPDATE Product SET ? WHERE prod_ID = ?) to update the product's information in the database. If the database operation encounters an error, the server responds with a 500 Internal Server Error; otherwise, it sends a success message indicating the product was updated, along with the number of affected rows.

```
app.listen(process.env.PORT, () => {  
  console.log(`Server listening on port: ${process.env.PORT}`);  
})
```

This code starts the server by telling the Express application (app) to listen for incoming network requests on a specific port. The port number is taken from the environment variable process.env.PORT, allowing flexibility to change the port without modifying the code (useful for deployment environments like cloud servers). When the server successfully starts, it executes a callback function that logs a message to the console indicating which port the server is running on (Server listening on port: <PORT>).

Public webservice

Main branch of Jelly Bunny

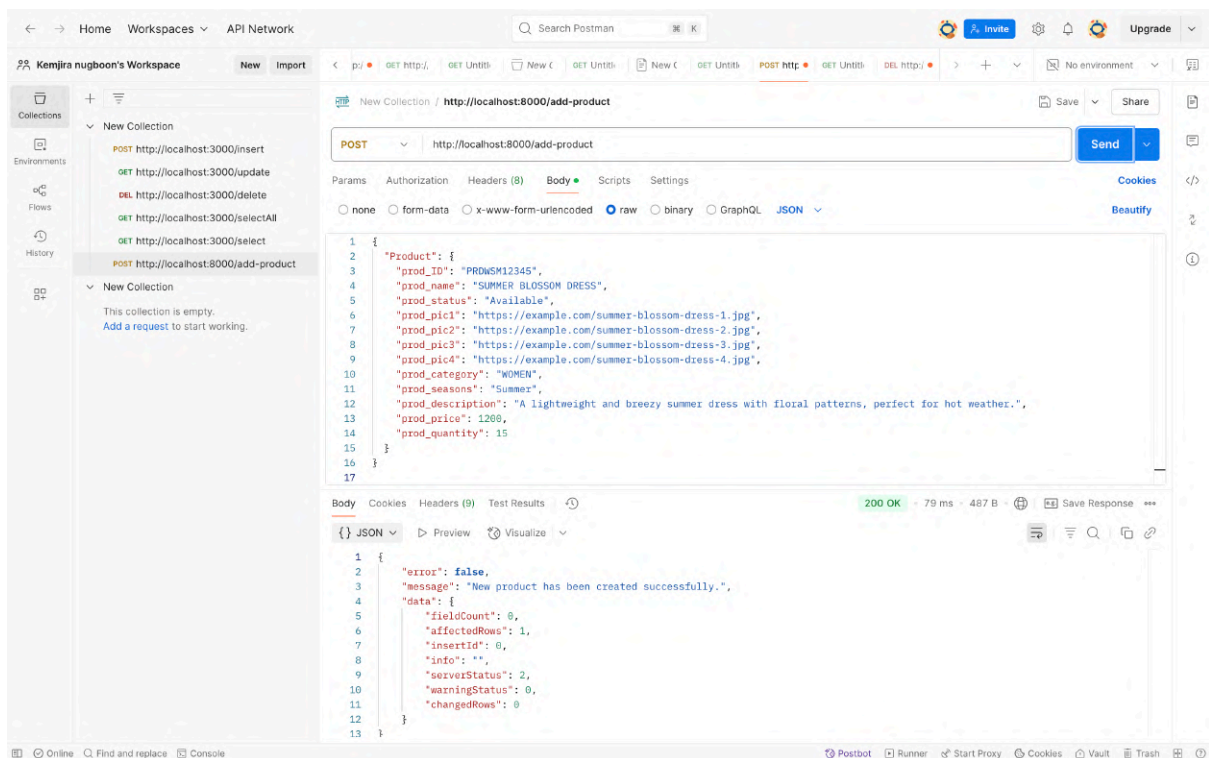
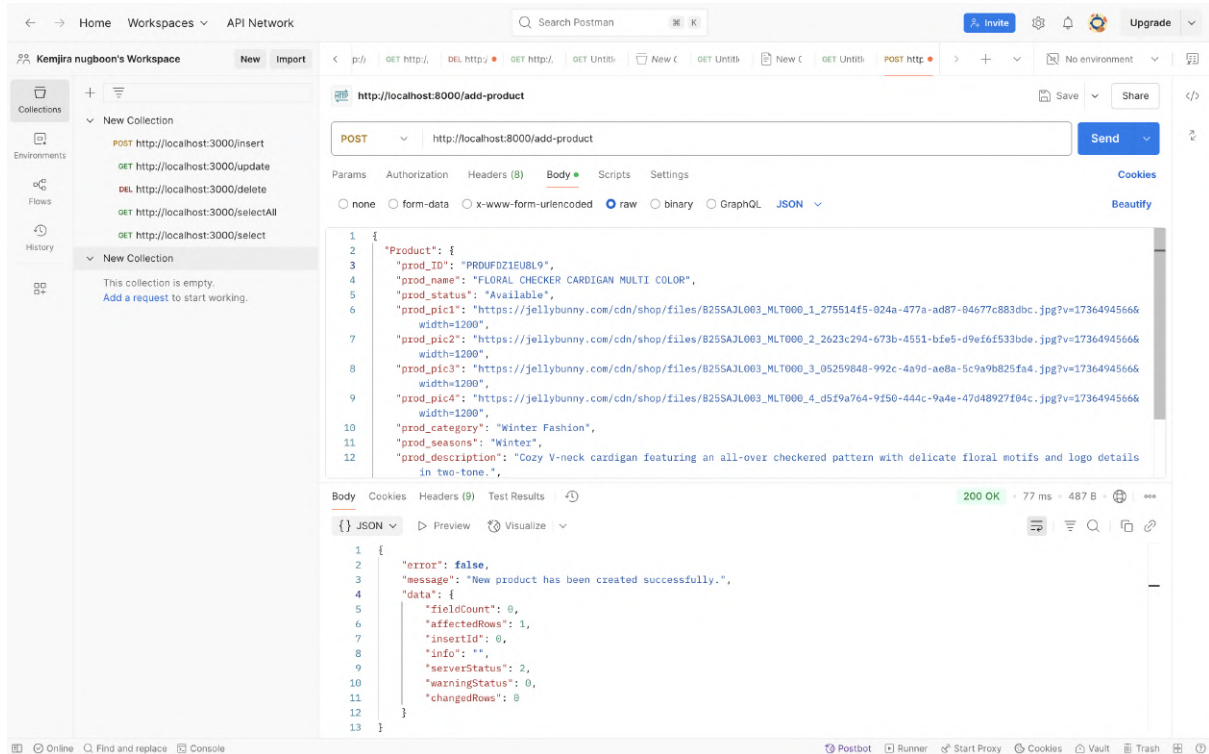


```
<h2>Main branch of Jelly Bunny</h2>  
<iframe width="100%" height="400" style="border:0" loading="lazy" allowfullscreen  
src="https://www.google.com/maps/embed/v1/place?key=AIzaSyCv0-1cJxCdo2jXibSkjQtN-DlvVIPVoZo6q=Jelly+Bunny+Zpell+Rangsit">  
</iframe>
```

The map is embedded in the **Home page** using an <iframe>, allowing users to view the store's physical location directly on the website. The embedded map provides a visual reference and enhances user experience, especially for users who wish to visit the store in person.

The testing results of web services using Postman or any program for testing web service

1. Insert Products



2. Update Products

The screenshot shows the Postman interface with a workspace named "Kemjira nugboon's Workspace". A new collection is created, and a POST request is defined with the URL `http://localhost:8000/update-product?prod_ID=PRDWSM12345`. The request body is a JSON object representing a product update:

```
1 {
2   "Product": {
3     "prod_name": "Updated Summer Blossom Dress",
4     "prod_status": "Available",
5     "prod_pic1": "https://example.com/updated-summer-dress-1.jpg",
6     "prod_pic2": "https://example.com/updated-summer-dress-2.jpg",
7     "prod_pic3": "https://example.com/updated-summer-dress-3.jpg",
8     "prod_pic4": "https://example.com/updated-summer-dress-4.jpg",
9     "prod_category": "WOMEN",
10    "prod_seasons": "Summer",
11    "prod_description": "An updated summer dress with fresh floral patterns.",
12    "prod_price": 1350,
13    "prod_quantity": 30
14  }
15 }
16 }
```

The response is a 200 OK status with a response time of 22 ms and a body size of 366 B. The response body is a JSON object:

```
1 {
2   "error": false,
3   "data": 0,
4   "message": "Product updated successfully"
5 }
```

The screenshot shows the Postman interface with the same workspace. A new collection is created, and a POST request is defined with the URL `http://localhost:8000/update-product?prod_ID=PRDUFZ1EU8L9`. The request body is a JSON object representing a product update:

```
1 {
2   "Product": {
3     "prod_name": "FLORAL CHECKER CARDIGAN MULTI COLOR (Updated)",
4     "prod_status": "Available",
5     "prod_pic1": "https://jellybunny.com/cdn/shop/files/B2SSAJL003_MLT000_1_275514f5-024a-477a-ad87-04677c883dbc.jpg?v=1736494566&
6     width=1200",
7     "prod_pic2": "https://jellybunny.com/cdn/shop/files/B2SSAJL003_MLT000_2_2623c294-673b-4551-bfe5-d9ef6f533bde.jpg?v=1736494566&
8     width=1200",
9     "prod_pic3": "https://jellybunny.com/cdn/shop/files/B2SSAJL003_MLT000_3_05259848-992c-4a9d-ae8a-5c9a9b825fa4.jpg?v=1736494566&
10    width=1200",
11    "prod_pic4": "https://jellybunny.com/cdn/shop/files/B2SSAJL003_MLT000_4_d5f9a764-9f50-444c-9a4e-47d48927f04c.jpg?v=1736494566&
12    width=1200",
13    "prod_category": "Winter Fashion",
14    "prod_seasons": "Winter",
15    "prod_description": "Cozy V-neck cardigan featuring an all-over checkered pattern with delicate floral motifs and logo details
16    in two-tone. (Updated version)",
17    "prod_price": 1890,
18  }
19 }
```

The response is a 200 OK status with a response time of 39 ms and a body size of 366 B. The response body is a JSON object:

```
1 {
2   "error": false,
3   "data": 0,
4   "message": "Product updated successfully"
5 }
```

3. Delete Products

The image displays two screenshots of the Postman application interface, showing the configuration and execution of DELETE API requests.

Top Screenshot:

- Collection:** New Collection / `http://localhost:8000/delete_product?prod_ID=PRDUF7PK07`
- Method:** DELETE
- URL:** `http://localhost:8000/delete_product?prod_ID=PRDUF7PK07`
- Query Params:**

Key	Value	Description
prod_ID	PRDUF7PK07	
- Response:** 200 OK - 19 ms - 358 B. Body (JSON):

```
{  "success": true,  "message": "Product deleted successfully"}
```

Bottom Screenshot:

- Collection:** New Collection / `http://localhost:8000/delete_product?prod_ID=PRDXR47OTBHBX`
- Method:** DELETE
- URL:** `http://localhost:8000/delete_product?prod_ID=PRDXR47OTBHBX`
- Query Params:**

Key	Value	Description
prod_ID	PRDXR47OTBHBX	
- Response:** 200 OK - 20 ms - 358 B. Body (JSON):

```
{  "success": true,  "message": "Product deleted successfully"}
```


4. Login

The screenshot shows the Postman interface with a workspace named "Kemjira nugboon's Workspace". The left sidebar contains a "Collections" panel with two "New Collection" entries. The first collection contains four requests: a POST request to "http://localhost:3000/insert", a GET request to "http://localhost:3000/update", a DEL request to "http://localhost:3000/delete", and a GET request to "http://localhost:3000/selectAll". The second collection contains three requests: a GET request to "http://localhost:3000/select", a DEL request to "http://localhost:8000/delete_prod...", and a POST request to "http://localhost:8000/add-product". The main panel shows a POST request to "http://localhost:8000/form-post". The request body is a JSON object:

```
{  "username": "alice.lin@gmail.com",  "password": "secure123"}
```

. The response is a 200 OK status with a response time of 26 ms and a body size of 345 B. The response body is a JSON object:

```
{  "error": false,  "message": "successful login"}
```

.

Invalid password

The screenshot shows the Postman interface with the same workspace and collections as the previous screenshot. The main panel shows a POST request to "http://localhost:8000/form-post". The request body is a JSON object:

```
{  "username": "alice.lin@gmail.com",  "password": "secure124"}
```

. The response is a 401 Unauthorized status with a response time of 82 ms and a body size of 385 B. The response body is a JSON object:

```
{  "error": true,  "message": "Invalid username or password. Please try again."}
```

.

Postman interface showing a successful POST request to `http://localhost:8000/form-post`. The request body is a JSON object:

```
1 {
2   "username": "grace.li@gmail.com",
3   "password": "gracepw"
4 }
```

The response is a 200 OK status with a JSON body:

```
1 {
2   "error": false,
3   "message": "successful login"
4 }
```

Invalid password

Postman interface showing an unsuccessful POST request to `http://localhost:8000/form-post` due to an invalid password. The request body is a JSON object:

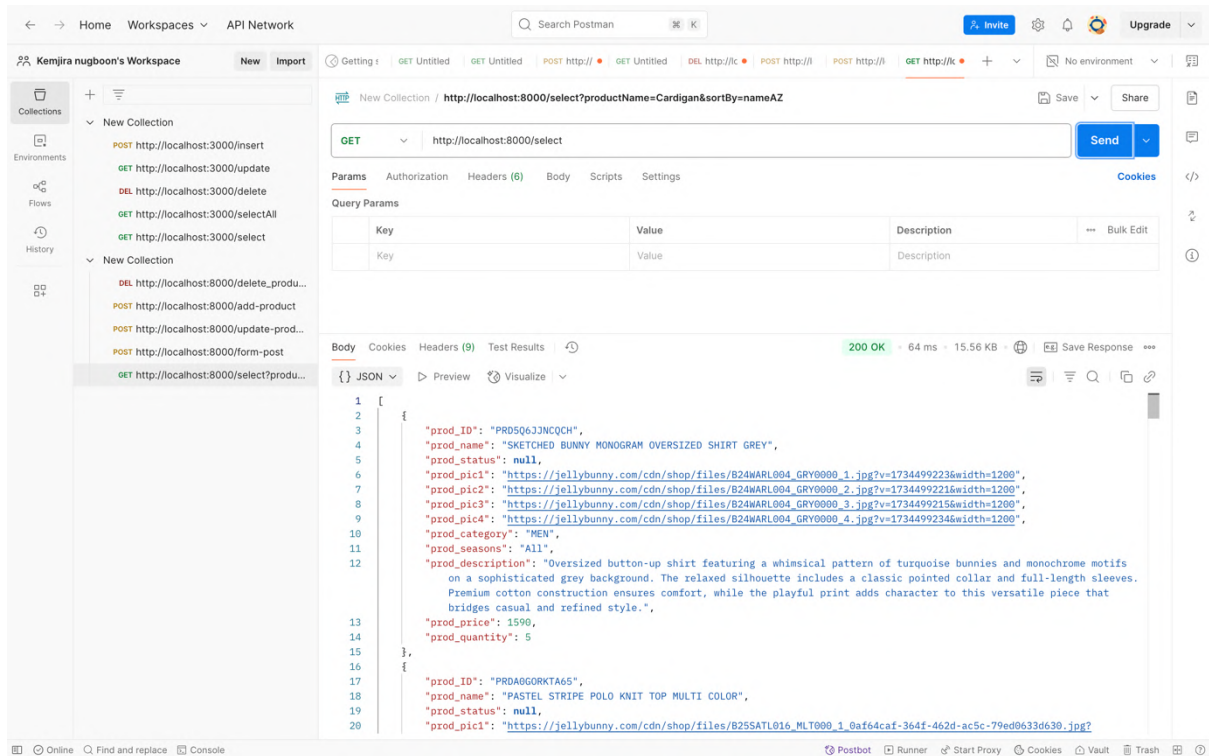
```
1 {
2   "username": "grace.li@gmail.com",
3   "password": "gracepw99"
4 }
```

The response is a 401 Unauthorized status with a JSON body:

```
1 {
2   "error": true,
3   "message": "Invalid username or password. Please try again."
4 }
```

5. Search

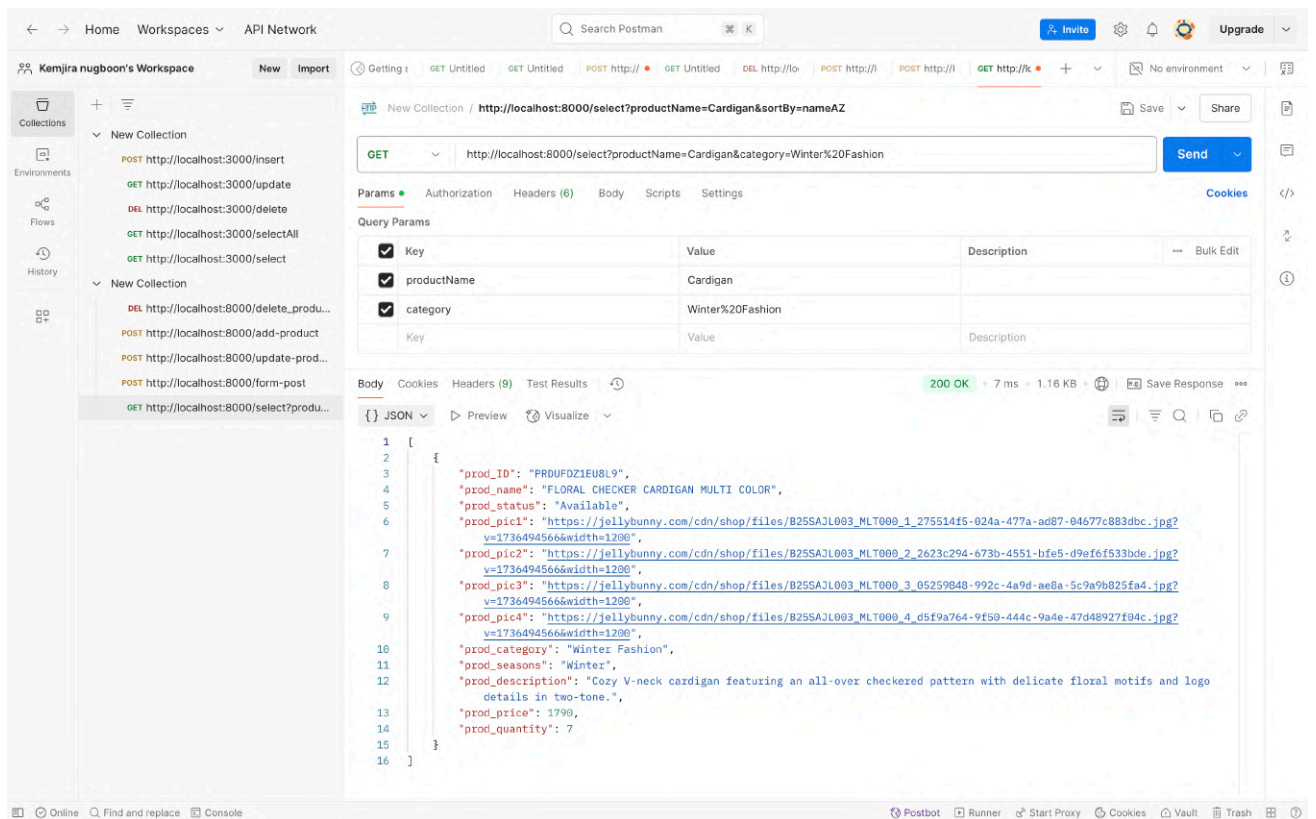
- No criteria search



The screenshot shows the Postman interface with a new collection named "http://localhost:8000/select?productName=Cardigan&sortBy=nameAZ". The request is a GET method to the URL "http://localhost:8000/select". The response is a 200 OK status with a response time of 64 ms and a body size of 15.56 KB. The response body is a JSON array containing two product objects.

```
1 [
2   {
3     "prod_id": "PRD5Q6JNCQCH",
4     "prod_name": "SKETCHED BUNNY MONOGRAM OVERSIZED SHIRT GREY",
5     "prod_status": null,
6     "prod_pic1": "https://jellybunny.com/cdn/shop/files/B24MARL004_GRY0000_1.jpg?v=1734499223&width=1200",
7     "prod_pic2": "https://jellybunny.com/cdn/shop/files/B24MARL004_GRY0000_2.jpg?v=1734499221&width=1200",
8     "prod_pic3": "https://jellybunny.com/cdn/shop/files/B24MARL004_GRY0000_3.jpg?v=1734499215&width=1200",
9     "prod_pic4": "https://jellybunny.com/cdn/shop/files/B24MARL004_GRY0000_4.jpg?v=1734499234&width=1200",
10    "prod_category": "MEN",
11    "prod_seasons": "All",
12    "prod_description": "Oversized button-up shirt featuring a whimsical pattern of turquoise bunnies and monochrome motifs on a sophisticated grey background. The relaxed silhouette includes a classic pointed collar and full-length sleeves. Premium cotton construction ensures comfort, while the playful print adds character to this versatile piece that bridges casual and refined style.",
13    "prod_price": 1590,
14    "prod_quantity": 5
15  },
16  {
17    "prod_id": "PRDAGCORKTA65",
18    "prod_name": "PASTEL STRIPE POLO KNIT TOP MULTI COLOR",
19    "prod_status": null,
20    "prod_pic1": "https://jellybunny.com/cdn/shop/files/B25SATL016_MLT000_1_0af64caf-364f-462d-ac5c-79ed0633d630.jpg?"
```

- Criteria search



The screenshot shows the Postman interface with a new collection named "http://localhost:8000/select?productName=Cardigan&category=Winter%20Fashion". The request is a GET method to the URL "http://localhost:8000/select?productName=Cardigan&category=Winter%20Fashion". The response is a 200 OK status with a response time of 7 ms and a body size of 1.16 KB. The response body is a JSON array containing one product object.

```
1 [
2   {
3     "prod_id": "PRDUFZ1EU8L9",
4     "prod_name": "FLORAL CHECKER CARDIGAN MULTI COLOR",
5     "prod_status": "Available",
6     "prod_pic1": "https://jellybunny.com/cdn/shop/files/B25SAJL003_MLT000_1_27551415-024a-477a-ad87-04677c883dbc.jpg?v=1736494566&width=1200",
7     "prod_pic2": "https://jellybunny.com/cdn/shop/files/B25SAJL003_MLT000_2_2623c294-673b-4551-bfe5-d9ef6f533bde.jpg?v=1736494566&width=1200",
8     "prod_pic3": "https://jellybunny.com/cdn/shop/files/B25SAJL003_MLT000_3_05259848-992c-4a9d-ae8a-5c9a9b825fa4.jpg?v=1736494566&width=1200",
9     "prod_pic4": "https://jellybunny.com/cdn/shop/files/B25SAJL003_MLT000_4_d5f9a764-9f50-444c-9a4e-47d48927f94c.jpg?v=1736494566&width=1200",
10    "prod_category": "Winter Fashion",
11    "prod_seasons": "Winter",
12    "prod_description": "Cozy V-neck cardigan featuring an all-over checkered pattern with delicate floral motifs and logo details in two-tone.",
13    "prod_price": 1790,
14    "prod_quantity": 7
15  }
16 ]
```

Postman interface showing a REST client request to `http://localhost:8000/select?productName=Cardigan&sortBy=nameAZ`. The request is a GET method. The response is a 200 OK status, indicating a successful retrieval of product data.

Query Params:

Key	Value	Description
productName	Cardigan	
sortBy	nameAZ	

Body (JSON):

```
{  "prod_id": "PRD0ZQLMFESC",  "prod_name": "CLASSIC CABLE KNIT CARDIGAN GREY",  "prod_status": null,  "prod_pic1": "https://jellybunny.com/cdn/shop/files/B25SAJL005_GRY900_1_69663496-d66f-4654-beca-4991b8352297.jpg?v=1736494556&width=1200",  "prod_pic2": "https://jellybunny.com/cdn/shop/files/B25SAJL005_GRY900_2_f9612414-b112-4ad6-94da-f7e8f11e90fa.jpg?v=1736494556&width=1200",  "prod_pic3": "https://jellybunny.com/cdn/shop/files/B25SAJL005_GRY900_3_9757a433-15a1-4c81-8ed2-36cfe17190ba.jpg?v=1736494556&width=1200",  "prod_pic4": "https://jellybunny.com/cdn/shop/files/B25SAJL005_GRY900_4_933924b7-64cd-4873-a4e5-71886c1daaf8.jpg?v=1736494556&width=1200",  "prod_category": "MEN",  "prod_seasons": "Winter",  "prod_description": "Classic cable knit cardigan featuring an intricate braided pattern throughout and a refined V-neckline. Adorned with antique-style buttons and a playful cat duo embroidery detail. The relaxed silhouette pairs functionality with style through practical front pockets and ribbed trim finishes. A versatile layering piece that brings both comfort and sophistication to any outfit.",  "prod_price": 1790,  "prod_quantity": 4},  {  "prod_id": "PRDUF21EUBL9",
```

Postman interface showing a REST client request to `http://localhost:8000/select?productName=Multi&season=Summer&sortBy=nameAZ`. The request is a GET method. The response is a 200 OK status, indicating a successful retrieval of product data.

Query Params:

Key	Value	Description
productName	MULTI	
season	Summer	
sortBy	nameAZ	

Body (JSON):

```
{  "prod_id": "PRDVLVWV7Wt58",  "prod_name": "CAMP FIRE TIME TIE DYE TEE MULTI COLOR",  "prod_status": null,  "prod_pic1": "https://jellybunny.com/cdn/shop/files/B25SATL007_MLT000_1_e6a8d9a3-7e6d-40eb-948d-25e54af6c63.jpg?v=1736494584&width=1200",  "prod_pic2": "https://jellybunny.com/cdn/shop/files/B25SATL007_MLT000_2_6a1bc96f-d884-4fcf-839f-68dc2a88f8e2.jpg?v=1736494584&width=1200",  "prod_pic3": "https://jellybunny.com/cdn/shop/files/B25SATL007_MLT000_3_cfc95f6-d244-4536-84b4-e9fd8d344058.jpg?v=1736494584&width=1200",  "prod_pic4": "https://jellybunny.com/cdn/shop/files/B25SATL007_MLT000_4_7136d942-184b-41b6-b236-db2a3ff02af.jpg?v=1736494584&width=1200",  "prod_category": "MEN",  "prod_seasons": "Summer",  "prod_description": "Relaxed-fit t-shirt featuring a charming campfire scene with whimsical animal characters set against a moody tie-dye background. The playful 'Camp Fire Time' graphic combines forest elements with cute illustrations. The oversized silhouette and unique dye pattern create an effortlessly cool aesthetic, perfect for a touch of adventure to casual ensembles.",  "prod_price": 1890,  "prod_quantity": 5},  {
```


HomeWorkspacesAPI Network

Search Postman

Invite

Upgrade

Kemjira nugboon's Workspace

NewImport

Getting tGET UntitledGET UntitledPOST http://...GET UntitledDEL http://...POST http://...POST http://...GET http://...

Collections

New Collection

POST http://localhost:3000/insert

GET http://localhost:3000/update

DEL http://localhost:3000/delete

GET http://localhost:3000/selectAll

GET http://localhost:3000/select

New Collection

DEL http://localhost:8000/delete_produ...

POST http://localhost:8000/add-product

POST http://localhost:8000/update-prod...

POST http://localhost:8000/form-post

GET http://localhost:8000/select7produ...

Environments

Flows

History

New Collection / http://localhost:8000/select?productName=Cardigan&sortBy=nameAZ

SaveShare

GEThttp://localhost:8000/select?season=Summer&sortBy=priceHighToLow

Send

ParamsAuthorizationHeaders (6)BodyScriptsSettingsCookies

Query Params

Key	Value	Description	Bulk Edit
season	Summer		
sortBy	priceHighToLow		
Key	Value	Description	

BodyCookiesHeaders (9)Test Results200 OK14 ms6.05 KBSave Response

JSONPreviewVisualize

```
1  [
2    {
3      "prod_id": "PRD05C9AGBRTA",
4      "prod_name": "TOM AND JERRY CHASE SCENE STRIPE BOWLING SHIRT YELLOW",
5      "prod_status": "NEW",
6      "prod_pic1": "https://jellybunny.com/cdn/shop/files/B25SARL008_YLW900_1_9cf9c9a6-8d69-4318-8bcd-3527cb1e6454.jpg?v=1737543486&width=1200",
7      "prod_pic2": "https://jellybunny.com/cdn/shop/files/B25SARL008_YLW900_2_4dd6b6bf-faf5-4a98-86b7-2c73dfc78940.jpg?v=1737543486&width=1200",
8      "prod_pic3": "https://jellybunny.com/cdn/shop/files/B25SARL008_YLW900_3_02837e66-286d-49f6-80ab-7a993e8f917c.jpg?v=1737543486&width=1200",
9      "prod_pic4": "https://jellybunny.com/cdn/shop/files/B25SARL008_YLW900_4_e7ef4f45-770d-4a4a-b6f0-b51e5c0b958a.jpg?v=1737543486&width=1200",
10     "prod_category": "WOMEN",
11     "prod_seasons": "Summer",
12     "prod_description": "Discover this retro-inspired bowling shirt featuring a playful Tom and Jerry chase scene on yellow pinstripes. The contrast white collar and relaxed fit are complemented by Jerry Chase detail on the chest pocket, creating a charming vintage-style piece.",
13     "prod_price": 1790,
14     "prod_quantity": 4
15   },
16   {
17     "prod_id": "PRDUW3R327YMC",
```

OnlineFind and replaceConsolePostbotRunnerStart ProxyCookiesVaultTrash

References

[1] Jelly Bunny, "Official Jelly Bunny Website," accessed Feb. 21, 2025. [Online]. Available: <https://jellybunny.com/en>

[2] Figma, "Community File," accessed Feb. 21, 2025. [Online]. Available: <https://www.figma.com/community/file/1219312065205187851>